1. Veiledet læring

1.1 Data

Variabler (statistikk) eller features (maskinlæring) er samme greien.

Kvalitative eller kvantitative variabeler:

- Kvalitative representerer kategorier.
  - Sjanger, aldersgruppe, type frukt ...
- Kvantitative representerer numeriske verdier:
  - Inntekt, alder, antall lyttere, areal, nedbørsmengde, temperatur...

Features benevnes x_n
Targets benevnes y_n
Indekserer med (i) som slik: $x^{(i)}$
Eksempel Titanic datasett:

Table 3: Titanic-datasettet etter at vi har valgt ut hvilke variable vi vil beholde, og innført notasjon.

	x_1	x_2	x_3	y
$\mathbf{x}^{(1)}$	3	male	22	0
$\mathbf{x}^{(2)}$	1	female	38	1
$\mathbf{x}^{(3)}$	3	female	26	1
$\mathbf{x}^{(4)}$	1	female	35	1
$\mathbf{x}^{(5)}$	3	male	35	0
$\mathbf{x}^{(6)}$	3	male		0
$\mathbf{x}^{(7)}$	1	male	54	0
$\mathbf{x}^{(8)}$	3	male	2	0
$\mathbf{x}^{(9)}$	2	male	27	1

Vi vil fjerne hullete data, encode kvalitative variabler og endre størrelsesorden på numeriske verdier. Deretter kan vi se på dataene!

1.2 Logistisk regresjon

Enkleste modellen er lineær regresjon:

$$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}\mathbf{x} + b$$

Der $\mathbf{w}$ er vektene og b er bias.

Kan gjøres om til sansynlighetsfordeling (mellom 0 og 1) ved å sette inn for #Sigmoid-funksjonen (også kjent som #aktiveringsfunksjon)

$$\sigma(z) = \frac{1}{1+e^{-z}} \implies y_{pred} = \frac{1}{1+e^{-(\sum_{i=1}^{n} w_i x_i + b)}}$$

1.2.1 Trening og tapsfunksjon

Funksjonen som brukes til å beregne målopnnåelse kalles for #tapsfunksjon. Forteller hvor nærme modellens prediksjon er den riktige verdien, eller target:

$$\mathcal{L}(y_{pred}, y)$$

Den tar inn modellprediksjonene y_{pred} og #targets y. **NB!** må være deriverbar (se 1.2.2 Gradient descent)
Eksempel link
Eksempel på funksjon for klassifisering av binær target:

$$L(y_{pred}, y) = y_{pred}^{y}(1 - y_{pred})^{1-y}$$

$$\implies L(\mathbf{w}, b|x, y) = \sigma(\mathbf{w}\mathbf{x} + b)^{y}(1 - \sigma(\mathbf{w}\mathbf{x} + b))^{(1-y)}$$

Vi vil finne en #likelihood og ikke en sannsynlighet fordi vi **ikke** har lyst til å endre dataene slik at sannsynligheten for target endrer seg. men heller endre modellen slik at den estimerte sannsynligheten passer med den virkelige (target).

Vi ønsker å ha likelihood for å estimere riktig være maksimal, så vi kan legge til et minustegn på tapsfunksjonen for å minimere denne. Vi bruker gjerne å ta logaritmen av av likelihood for å få **log-likelihood** (Minimere tap == maksimere Likelihood). Dette kalles for #cross-entropy-loss.

Eksempel på tapsfunksjon: #binary_cross-entropy-loss
Vektet tapsfunksjon: Maksimere likelihood for riktig kategorisering av dataene

$$\mathcal{L}(y_{pred}, y) = -\omega_1 y \ln y_{pred} - \omega_0(1 - y)\ln(1 - y_{pred})$$

Der ω_0 og ω_1 er vekter for klassene, se Vektet tapsfunksjon.
y er target og vi kan bruke y_{pred} fra formelen her.

Vi kan nå skrive om modellen definert av parametrene $\mathbf{w}$ og b for å finne parametrene som minimerer tapet i gjennomsnitt over alle datapunktene (x). Dette kaller vi for å **trene modellen**. Vi får da at vi skal maksimere log-likelihooden for parametrene som vi definerer:

$$\theta = (\mathbf{w}, b) \implies \hat{\theta} = arg_\theta \min \frac{1}{N}\sum_{i=1}^{N}\mathcal{L}(f(\mathbf{x}^{(i)}; \theta), y^{(i)})$$

Detta kalles **conditional maximum likelihood estimation**, og kan løses med bruk av gradient descent.

1.2.2 Gradient descent

Når vi ser på funksjonen vår beveger vi oss i rommet spent ut av verdiene θ, fire verdier gir et fire dimensjonalt rom. Vi vil finne minimumspunktet til funksjonen uten at vi kjenner formen til den. **NB!** så lenge tapsfunksjonen er konveks, så vil vi alltid kunne finne lokalt minium med gradienten.

I starten av treningen har alle parametrene en tilfeldig verdi, og vi oppdaterer verdiene iterativt, gjennom flere steg. Gradienten til tapsfunksjonen peker i retning av økende tap, og siden vi ønsker å minke tapet må vi bevege oss i motsatt retning. Dette representerer vi gjennom et minustegn, og regelen for parameteroppdateringen kan skrives som følger:

$$\theta^{t+1} = \theta^t - \eta \frac{\partial}{\partial \theta} \mathcal{L}(f(\mathbf{x}; \theta), y)$$

Der η er #læringsraten, som bestemmer hvor mye hvert steg t skal korrigere til parameterverdien. **NB!** η eller læringsraten er en parameter for å justere treningen til modellen, men er **ikke** en del av modellen! Dette kaller vi for en #hyperparameter, som definerer læringsprosessen, men ikke modellen. Den kan vi leke med/justere mye for å optimalisere, men generelt ønsker små steg for å ikke hoppe over minimumspunktet men også ikke fordi da vil modellen bruke lang tid på å læres.

Framgangsmåten vil da bli å finne gradienten av tapsfunksjonen, med hensyn til vær av parametrene. Eksempel med titanic modellen:

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial w_j} &= \frac{\partial}{\partial w_j} \left[-y \ln \sigma\left(\mathbf{wx} + b\right) - (1-y) \ln\left(1 - \sigma(\mathbf{wx} + b)\right) \right] \\
&= -y \frac{1}{\sigma(\cdot)} \frac{\partial}{\partial w_j} \sigma(\mathbf{wx} + b) + (1-y) \frac{1}{1 - \sigma(\cdot)} \frac{\partial}{\partial w_j} \left(1 - \sigma(\mathbf{wx} + b)\right) \\
&= \frac{-y\left(1 - \sigma(\cdot)\right) + (1-y)\sigma(\cdot)}{\sigma(\cdot)(1 - \sigma(\cdot))} \frac{\partial}{\partial w_j} \sigma(\mathbf{wx} + b) \\
&= \frac{\sigma(\cdot) - y}{\sigma(\cdot)\left(1 - \sigma(\cdot)\right)} \sigma(\cdot)\left(1 - \sigma(\cdot)\right) \frac{\partial}{\partial w_j} (\mathbf{wx} + b) \\
&= \left(\sigma(\mathbf{wx} + b) - y\right) x_j \\
&= (y_{\text{pred}} - y) x_j \\
\frac{\partial \mathcal{L}}{\partial b} &= \dots \\
&= \left(\sigma(\mathbf{wx} + b) - y\right) \frac{\partial}{\partial b} (\mathbf{wx} + b) \\
&= \left(y_{\text{pred}} - y\right).
\end{aligned}$$

1.2.3 Trening

En måte å strukturere #læringsalgoritme på:

PYTHON

```python
class LogisticRegression:
    def __init__(self, learning_rate=0.1, epochs=1000):
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.weights,
        self.bias = None, None
        self.losses, self.train_accuracies = [], []

    def sigmoid_function(self, x):

    def _compute_loss(self, y, y_pred):

    def compute_gradients(self, x, y, y_pred):

    def update_parameters(self, grad_w, grad_b):

    def accuracy(true_values, predictions):
        return np.mean(true_values == predictions)

def fit(self, x, y):
    self.weights = np.zeros(x.shape[1]) #x.shape = datapunkter, features
    self.bias = 0

    # Gradient Descent
    for _ in range(self.epochs):
        lin_model = np.matmul(self.weights, x.transpose()) + self.bias
        y_pred = self._sigmoid(lin_model)
        grad_w, grad_b = self.compute_gradients(x, y, y_pred)
        self.update_parameters(grad_w, grad_b)

        loss = self._compute_loss(y, y_pred)
        pred_to_class = [1 if _y > 0.5 else 0 for _y in y_pred]
        self.train_accuracies.append(accuracy(y, pred_to_class))
        self.losses.append(loss)

def predict(self, x):
    lin_model = np.matmul(x, self.weights) + self.bias
    y_pred = self._sigmoid(lin_model)
    return [1 if _y > 0.5 else 0 for _y in y_pred]
```

Når skal bruke koden, bør en **ikke** bruke hele datasettet til å trene modellen. Del dataen opp i #testdata og #treningsdata. Treningsdataene brukes til parametertilpasning, mens testdataene ikke røres før modellen er ferdig tilpasset, og brukes for å evaluere modellen før denne tas i bruk. Bruk gjerne `train_test_split` fra biblioteket `scikitłearn`.

Eksempel på kjøring av kode:

```python
# Training
train_epochs = 30

# Initialize and train the model
log_reg = LogisticRegression_(learning_rate=0.01, epochs=train_epochs)
log_reg.fit(X_train, y_train)

# Make predictions
predictions = log_reg.predict(X_test)
```

1.2.4 Evaluering

Lurt å plotte tapet (loss) mot treffsikkerhet (accuracy) for å se treningsprosedyren er god.

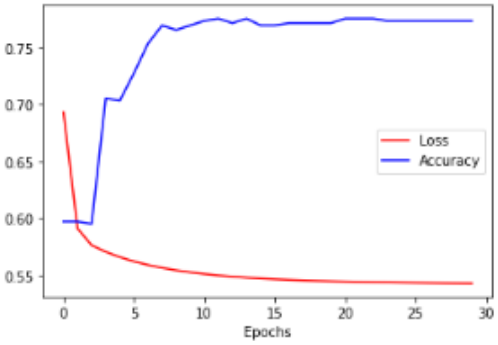


Figure 4: Eksempel på plott som viser tap (rød) og treffsikkerhet (blå) over epokene i en treningsprosedyre.

Mange metrikker baseres på #confusion_matrix som brukes ofte for å evaluere klassifiseringsmodeller.

		Predicted	
	Total (P+N)	Negative (PN)	Positive (PP)
True	Negative (N)	True negative	False positive
	Positive (P)	False negative	True positive

Figure 5: Confusion matrix for binær klassifisering.

Dette kan vi bruke til å regne ut flere matrikker som i tabellen under:

Table 4: Et utvalg metrikker vi kan beregne basert på verdiene i confusion matrix.

Betegnelse	**Uttrykk**	**Alternative navn**
True Positive Rate (TPR)	$\frac{TP}{P} = \frac{TP}{TP+FN}$	Sensitivitet, recall, sannsynlighet for deteksjon
True Negative Rate (TNR)	$\frac{TN}{N} = \frac{TN}{TN+FP}$	Spesifisitet
False Positive Rate (FPR)	$\frac{FP}{N} = \frac{FP}{FP+TN}$	Sannsynlighet for falsk alarm, (1 - spesifisitet)
Positive Predictive Value (PPV)	$\frac{TP}{TP+FP}$	Precision

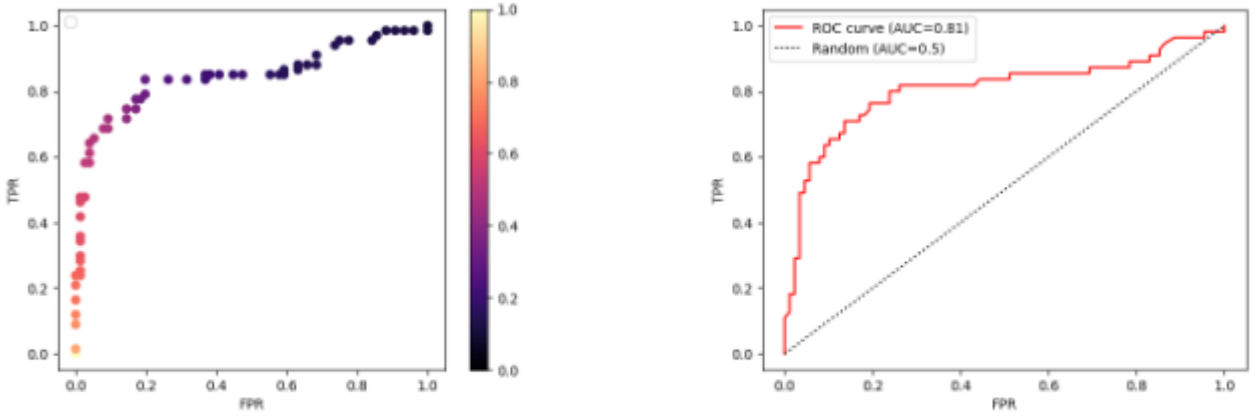
1.2.5 Klassifiseringsterskel

Enkel måte å justere på klassifiseringsterskelen er å endre linjen i `fit` og `predict` funksjonene:

```
[1 if _y > K else 0 for _y in y_pred]
```

Der `K` er klassifiseringsterskelen. Hvis en vil være sikker på at modellens predikasjon er riktig, vil en velge en høyere verdi for klassiferingsterkselen som 0.7.

Det er vanlig å plotte **FPR** (False Positive Rate) og **TPR** (True Positive Rate) som funksjon av klassifiseringsterskelen. Dette gir oss **Receiver Operating Characteristic** (#ROC)- kurven. Området under kurven til ROC kaller vi for #AUC (area under curve) som vi ønsker skal være større enn 0.5 (tilsvarer tilfeldig gjetting). ROC AUC representer sannsynligheten for at modellen vil predikere en høyere verdi for et tilfeldig valgt posititvt datapunkt enn for et tilfeldig valgt negativt datapunkt.

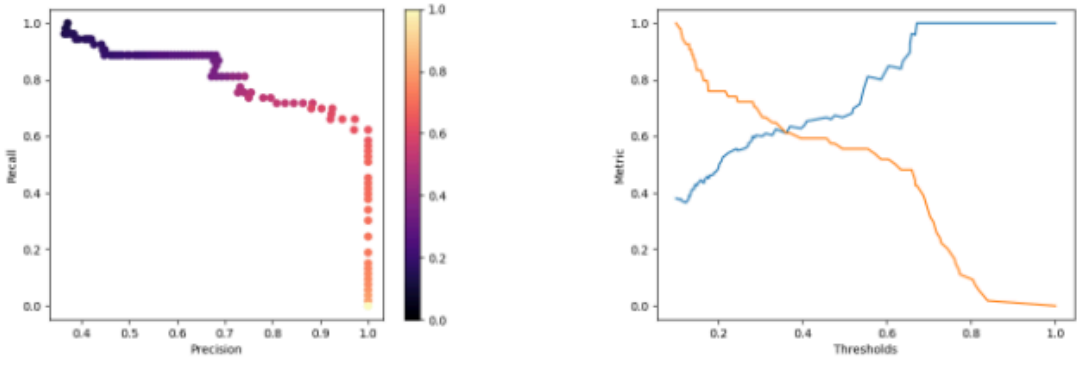


(a) (b)

Figure 6: ROC-kurven, som viser FPR og TPR for ulike klassifiseringsterskler.

Ingen tradeof mellom TPR og FPR, men alltid en tradeoff med metrikkene en ønsker å ha høy:

- En annen vanlig kurve å studere er den som viser precision (TPR) og recall (PPV).



(a) (b)

Figure 7: (a) Precision-recall-kurven , og (b) henholdsvis precision- og recall-kurven, for ulike klassifiseringsterskler.

Presisjon: Angir andel korrekt predikert positive per predikert positiv.

- Høy presisjon svarer til lavt relativt antall falske alarmer.
- Viktig om falske alarmer er dyrt, som å rykke ut til brann.

Recall: Angir andell korrekt predikert positive per totalt psoitive.

- Høy recall svarer til lavt antall missed cases.
- Viktig om kritisk å ikke treffe på en case, som med detektere kreft.

1.2.6 Kort om entropi og cross entropy loss

I informasjonsteori er entropi (også kalt Shannon entropy) en måte å kvantisere usikkerheten eller mengden informasjon tilgjengelig i en variabels mulige tilstander:

$$H(p) = -\sum_i p(x_i) \log p(x_i)). \tag{28}$$

I tilfellet med Titanic-dataene representerer $p(x_i)$ her sannsynligheten for `Survived`. Kryss-entropien (cross entropy) representerer da forskjellen mellom to ulike sannsynlighetsfordelinger p og q:

$$H(p, q) = -\sum_i p(x_i) \log q(x_i)) \,. \tag{29}$$

Hvis vi sammenligner dette med #tapsfunksjon og #cross-entropy-loss , ser vi at den ene sannsynlighetsfordelingen representerer labels i datasettet, mens den andre representerer modellens prediksjoner. Tapsfunksjonen forteller oss altså om forskjellen mellom de to fordelingene, og siden målet vårt er å lage en klassifiseringsmodell, ønsker vi at forskjellen mellom de to fordelingene er minst mulig. Vi kan generalisere utrykket over til:

$$\mathcal{L}(\hat{y}, y) = -\sum_i y_i \log(\hat{y}_i),$$

hvor summen går over alle klassene i.

1.2.7 Bayes' teorem

Vi kan bruke Bayes' teorem til å regne:

$$p(C_k|x) = \frac{p(x|C_k)p(C_k)}{p(x)}$$

Vi får da at:

- $p(C_k)$ er $a\ priori$ sannsynligheten for klasse, altså fordelingen av klassene i treningsdataene. Denne omtales som prior, fordi det er sannsynligheten for **klassetilhørighet** vi kan estimere uten å vite noe om det aktuelle datapunktet (legg merke til at x ikke forekommer i dette leddet).

- $p(x)$ er fordelingen av selve dataene, uavhengig av klasse (legg merke til at C_k ikke forekommer i dette leddet). Dette er den faktiske sannsynlighetsfordeligen av alle feature-verdiene for alle dataene, og den har like mange dimensjoner som datasettet har features. Dette leddet omtales som evidens.
- $p(x|C_k)$ er den betingede sannsynlighetsfordelingen av dataene gitt klassen, og det er en likelihood. Det er altså en egen sannsynlighetsfordeling – med like mange dimensjoner som datasettet har features – for hver klasse.

Vi kan regne $p(C_k)$, og for titanic casen kan vi se at $p(x)$ vil være vanskelig å bergne, men at den bidrar like mye til begge klassene så vi kan se vekk i fra den og få at:

$$p(C_k) \propto p(x|C_k)p(C_k)$$

Hvis vi antar at verdiene til featuresene er uavhengige av hverandre for alle features, så kan vi skrive dette om til Naiv Bayes:

$$p(C_k|x) \propto p(C_k) \prod_{i=1}^{n} p(x_i|C_k)$$

Som gir at den tilsvarende sannsynligheten for klassetilhørighet:

$$\hat{y} = \arg_{k\,\epsilon[1,\ldots,n]} \max p(C_k) \prod_{i=1}^{n} p(x_i|C_k)$$

1.2.8 Dimensjonsforbannelsen

Om vi ønsker å estimere en d-dimensjonal fordeling, er antallet datapunkt som må samles inn avhengig av d. Hvis vi tenker oss at vi estimerer fordelingen ved hjelp av et histogram med 10 bins (per dimensjon) hvor hver bin har 10 datapunkter (100 datapunkter per dimensjon). Et mye brukt datasett #MNIST har 784 dimensjoner, og det finnes ca 10^8atomer i universet.

Dimensjoner d	Bins	Datapunkter
1	10	10^2
3	10^3	10^4
10	10^{10}	10^{11}
784	10^{784}	10^{785}

1.2.9 Trening på ubalansert data

Om vi har mye flere datapunkter for forskjellige klasser, ender vi opp med å få **underrepresentert** og **overrepresentert** klasser. Hvis forskjellen mellom klasser blir for stor, vil modellen belønnes om den bare predikerer at alle datapunktene tilhører den overrepresenterte klassen som gir en dårlig modell. For å justere skjevfordelingen mellom klassene kan vi endre på klassifiseringsterskelen som i utrykket:

```
y_pred = [1 if _y > K= 05 else 0 for _y in y_pred]
```

fra 0.5 til en verdi som gir modellen høyere verdi på de andre metrikkene. Kan bruke precision-recall plottet som et ugangspunkt. Å maksimere en metrikk kommer på bekostning av andre metrikker og å justere terkselen forbedrer **ikke** modellen: det endrer kun hvordan vi forholder oss til modellens prediksjoner. Vi kan også endre dataene eller tapsfunksjonen.

Resampling

Undersampling:

Her trekker vi like mange datapunkter fra den overrepresenterte klassen som vi har tilgjengelig i den underrepresenterte klassen. Da ender vi opp med et datasett bestående av like mange datapunkter fra hver klasse, men potensielt veldig få datapunkter totalt. Dette kan føre til at modellen som trenes på dataene undertilpasser (underfit).

Oversampling:

Her kopierer vi instanser fra den underrepresenterte klassen, inntil vi har like mange datapunkter fra den underrepresenterte som fra den overrepresenterte klassen. Da ender vi også opp med like mange datapunkter fra hver klasse, men potensielt mange duplikater fra den underrepresenterte klassen. Dette kan føre til at modellen som trenes på dataene overtilpasser (overfit)

Vektet tapsfunksjon

Vi kan også fortelle modellen hvilken av klassene som er ekstra viktig ved å straffe feilprediskjoner (høyere tap) på den viktige klassen, relativt til de andre klassene. F.eks med #binary_cross-entropy-loss. Vi ser at for de to leddene, bidrar kun det første leddet til tapet når y=1 og det andre leddet når y=0. Ved å sette inn vektene w_1 w_0 for henholdsvis første og andre leddet kan vi gjøre at feilprediksjoner for den ene vil gi ut høyere tap enn den andre.

1.3 Beslutningstrær

1.3.1 Noder

#Root-node eller rotnoden er starten på beslutningstreet av dataene med alle featuresene som er splittet i en trestruktur.

Under har vi beslutningsnoder (de)cision nodes) som begge har kriterier for å splitte dataene (splitting criteria). Alle noder som splitter dataene er enten rotnoden eller beslutningsnoder.

Nederst i treet er løvnodene (leaf nodes) som ikke splitter dataene, som inneholder predikert verdi for datainstansen som ble sendt gjennom treet.

Beslutningstrær består av trestumper (tree stumps) som igjen består av en rotnode og n løvhoder, for n mulige utfall.

Treningsdataene brukes for å finne ut hvilke trestumper (og tilhørende beslutningskriterier) som bør settes sammen for å lage treet. Når vi bygger beslutningstrær ønsker vi alltid å velge det splitt-kriteriet som lar oss ta beslutningen tidligst mulig, altså reduserer usikkerheten mest mulig. Redusert usikkerhet er endringen i usikkerhet etter sammenliknet med før splitt. I hovedsak brukes følgende tre metrikker for å måle **hvor** mye et splitt-kriterium (feature og verdi) reduserer usikkerheten:

- Log loss (Se #tapsfunksjon og #cross-entropy-loss)
- Gini impurity
- Entropi

1.3.2 Gini impurity

Gini-urenheten er et tall i [0, 0.5] som angir sannsynligheten for at et nytt, tilfeldig datapunkt feilklassifiseres hvis det gis et tilfeldig label i henhold til klassedistribusjonen i datasettet. Gitt et datasett D bestående av datapunkter fra k klasser, med sannsynlighet pi for at en instans tilhører klassen i ved en gitt node, er datasettets Gini-urenhet:

$$\text{Gini}(D) = 1 - \sum_{i=1}^{k} p_i^2$$

Intuitivt: Du har en pose med kuler i ulike farger, hvor farge representerer klasse. Gini-urenheten måler hvor sannsynlig det er at du gjetter feil farge på en tilfeldig trukket kule, hvis du gjetter at kulens farge følger distribusjonen av farger i posen.
Lav Gini-urenhet representerer scenariet der de fleste kulene har samme farge, slik at det er lav sannsynlighet for å gjette feil farge på en tilfeldig trukket kule. Datasettet regnes da å ha lav urenhet.
Høy Gini-urenhet representerer motsatt scenario, der klassene er forholdsvis likt representert, og det er høy sannsynlighet for å gjette feil farge på en tilfeldig trukket kule. Datasettet regnes da å ha høy urenhet.

Hvis dataset D splittes på feature f til to subsett D_1 og D_2 med henholdsvis n_1 og n_2 datapunkter, har vi

$$\text{Gini}_f(D) = \frac{n_1}{n}\text{Gini}(D_1) + \frac{n_2}{n}\text{Gini}(D_2)$$

1.3.3 Entropi

Gitt en sannsynlighetsfordeling over k klasser, er sannsynligheten for hver klasse p_i. Entropien til fordelingen er da gitt ved:

$$(p_1, \ldots, p_k) = -\sum_{i=1}^{k} p_i \log(p_i)$$

Den maksimale entropien til en binær fordeling er 1, som svarer til lik sannsynlighet per klasse. Den minimale entropien er 0, som svarer til at alle datapunktene tilhører samme klasse.

Før vi splitter på en feature har vi et datasett D med p positive og n negative instanser. Dette svarer til en binær distribusjon der positiv klasse har sannsynlighet $\frac{p}{p+n}$, og negativ klasse har sannsynlighet $\frac{n}{p+n}$, og entropien er

$$\mathrm{I}_{\text{før}} = \mathrm{I}\left(\frac{p}{p+n}, \frac{n}{p+n}\right). \tag{43}$$

Etter at treet splitter på en feature, har vi to nye datasett, D_1 og D_2 (ett på hver side av splitten). Den forventede entropien etter denne splitten er generelt

$$\mathbb{E}[\mathrm{I}_{\text{etter}}] = \sum_{i=1}^{s} \frac{p_i + n_i}{p+n} \mathrm{I}\left(\frac{p_i}{p_i+n_i}, \frac{n_i}{p_i+n_i}\right). \tag{44}$$

hvor s angir antall splitt, i vårt tilfelle $s = 2$, slik at $i = 1, 2$. Vi er interessert i forskjellen i entropi, altså entropien før og etter en valgt splitt:

$$\Delta\mathrm{I} = \mathrm{I}_{\text{før}} - \mathbb{E}[\mathrm{I}_{\text{etter}}] = \mathrm{I}\left(\frac{p}{p+n}, \frac{n}{p+n}\right) - \sum_{i=1}^{s} \frac{p_i + n_i}{p+n} \mathrm{I}\left(\frac{p_i}{p_i+n_i}, \frac{n_i}{p_i+n_i}\right), \tag{45}$$

og vi bør velge den splitten som gir størst endring i entropi. **Oppgave:** Vi har en beslutningsnode bestående av $p = 9$ positive og $n = 5$ negative datapunkter. Vis at $I_{\text{før}} = 0.94$. Anta deretter at vi splitter datasettet slik at vi får to nye datasett med henholdsvis $(p_1 = 4, n_1 = 5)$ og $(p_2 = 5, n_2 = 0)$. Regn ut endringen i entropi (svaret står rett under, men prøv selv).

$$\mathbb{E}[\mathrm{I}_{\text{etter}}] = \frac{4+5}{9+5}\mathrm{I}\left(\frac{4}{9}, \frac{5}{9}\right) + \frac{5}{9+5}\mathrm{I}\left(\frac{5}{5}, \frac{0}{5}\right) = -\frac{9}{14}\left(\frac{4}{9}\log_2\frac{4}{9} + \frac{5}{9}\log_2\frac{5}{9}\right) + 0 \approx 0.64. \tag{46}$$

1.3.4 Bygge beslutningstrær

For å bygge beslutningstrær brukes som oftest biblioteket `sklearn.tree.DecisionTreeClassifier`. Den lar en velge mellom de splitt-kriteriene Gini impurity, entropy og log loss.

Gini impurity og entropy er oppfører seg likt, hovedforskjellen er entropy har en ekstra logaritme i kjøretid. Gini impurity er mindre beregningstungt og default i `sklearn.tree.DecisionTreeClassifier`.

Utover valg av beslutningskriterier, må vi bestemme:

- Når skal vi slutte å splitte, selv om nederste node har instanser fra begge klassene?
- Hva gjøres med løvnoder med instanser fra begge klassene?
  Kan oppstå for 3 tilfeller vi må slutte å splitte:

1. Når det ikke finnes flere features å splitte på. Dvs har splittet på alle tilgjengelige features, men har ikke laget løvnoder som tilordner alle treningsdatapunktene til riktig klasse.
2. Når det ikke finnes flere datapunkter å teste. Alle kombinasjoner av features som er **tilgjengelig i dataene** har blitt testet, men alle tenkelige kombinasjoner av features ikke er testet.
3. Når treet har nådd en predifinert maksimal dybde fra hyperparameter vi velger før begynner å bygge treet.

Etter at treet er bygget vil vi sannsynligvis ha løvnoder som inneholder treningsdatapunkter fra begge klasser. For å bestemme hvilken beslutning en slik node kan ta har vi flere muligheter, hvorav de vanligste er å returnere:

1. Den dominante klassen i noden, altså label tilsvarende den dominante klassen fra treningsdataene i løvnoden.
2. Et tilfeldig trukket label fra treningsdataene, altså **a priori**-sannsynligheten.

Vi kan nå sette sammen alt vi har vært gjennom til en algoritme som rekursivt bygger et beslutningstre, se pseudokode under.

```
def build_decision_tree(data, features, depth):
    if (all data in same class):
        return class label
    elif (no features left to test) or (depth >= max_depth):
        return 1 if p/(p+n) > n/(p+n) else 0
    elif (no data left to test):
        return (dominant class in parent node)
    else:
        choose best_feature f for split
        partition data based on f
        for each data partition:
            build_decision_tree(data partition, features - best_feature, depth+1)
```

I forelesningen brukes datasettet Palmer Penguins som eksempel. Dette datasettet er superpopulært innenfor maskinlæring, og kan brukes til mye mer enn å lage et beslutningstre som finner pingvinkjønn basert på kroppsekt og flipper length. Hvis du vil reprodusere noen av eksemplene, finner du en `.csv`-fil på internett som inneholder dataene, og kommer i gang med koden under.

```
df = pd.read_csv("data/penguins_lter.csv")
df = df[["Flipper Length (mm)" , "Body Mass (g)", "Sex"]].dropna()
label_encoder = LabelEncoder()
df["Sex"] = label_encoder.fit_transform(df["Sex"])

X_data = df[["Flipper Length (mm)" , "Body Mass (g)"]].values
y_data = df['Sex'].values

X_train, X_test, y_train, y_test = train_test_split(X_data, y_data, test_size=0.3)

clf = tree.DecisionTreeClassifier(max_depth=7, criterion="log_loss")
clf.fit(X_train, y_train)
```

Og så er det bare å kose seg. For å visualisere beslutningsflaten slik vi så på i forelesning, kan du bruke `from sklearn.inspection import DecisionBoundaryDisplay`.

1.4 Regresjon

1.4.1 Data og tapsfunksjon

Prediksjon til kontinuerlige verdier kalles **regresjon**. Generelt: estimering av en (eller flere) kontinuerlige verdier. Enkleste tilfellet er lineære regresjonsmodellen:

$$f(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots \theta_n x_n$$

Kan bruke igjen mye av kode fra klassifisering, og har igjen en bias θ_0 og vekter/parameter θ_i for hver dataegenskap x_i. Vi bruker igjen #gradient_descent for å optimalisere parametrene.

Vanligste tapsmodellen innen regresjon er #mean_squared_error:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}[y^{(i)} - f(x^{(i)})]^2$$

Her er $y^{(i)}$ er target for datapunkt $\mathbf{x}^{(i)}$, f er modellen og summen (gjennomsnittet) går over alle N instansene (radene) i datasettet. Liten MSE gir god prediksjon og stor gir dårlig. Gradient descent for et datasett med en feature x_1 gir oss:

$$\frac{\partial \mathcal{L}}{\partial \theta_0} = \frac{1}{N}\sum_{i=1}^{N}(\theta_1 x^{(i)} + \theta_0 - y^{(i)})$$

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = \frac{1}{N}\sum_{i=1}^{N} x^{(i)}(\theta_1 x^{(i)} + \theta_0 - y^{(i)})$$

Oppdateringsregel til parametrene i regresjonsmodellen blir da:

$$\theta_0 \leftarrow \theta_0 - \eta\frac{1}{N}\sum_{i=1}^{N}(f(x^{(i)}) - y^{(i)}) \quad \theta_1 \leftarrow \theta_1 - \eta\frac{1}{N}\sum_{i=1}^{N} x^{(i)}(f(x^{(i)}) - y^{(i)})$$

1.4.2 Bias og varians

Skille mellom bias leddet θ_0 og bias til en modell som forskjellen mellom den sanne verdien vi prøver å estimere, og forventningsverdiene til estimatet verdien:

$$\text{Bias}(y, \hat{y}) = \mathbb{E}[\hat{y} - y] = \mathbb{E}[\hat{y}] - y$$

Der $\hat{y}$ er en estimator (y_pred) beregnet på et tilfeldig utvalg datapunkter.
Annet nyttig er varians, som gir spredningen prediksjonene har fra gjennomsnitssverdien:

$$\text{Var}[\hat{y}] = \mathbb{E}\left[(\hat{y} - \mathbb{E}[\hat{y}])^2\right]$$

Dette er nyttig da

- høy varians ofte indikerer at modellen er i overkant sensitiv til variasjoner i data, som tyder på overtilpasning (overfit),
- lav varians ofte indikerer at modellen predikerer for nært gjennomsnittsprediksjonen, og ikke gjør tilstrekkelig nytte av informasjonen i features, som tyder på undertilpasning (underfit).

For en og samme modell finnes det en avveining mellom bias og varians, kjent som #bias_variance_tradeoff. Kan utlede for MSE-tapsfunksjonen (se hefte):

$$\mathbb{E}[y, \hat{y}] = \text{Bias}(y, \hat{y})^2 + \text{Var}(\hat{y}) + \sigma^2$$

Dette er viktig for å innse at en modells tap består av en komponent fra bias og en fra varians (pluss irredusibelt støy). Dette skaper "the bias variance dilemma" fordi å minske disse kildene til prediksjonsfeil fører til en konflikt, kan ikke ha begge for lave ettersom det gir en dårlig modell. (Lav bias gir veldig generalisering på treningsdataene slik at ikke bruker features ordentlig, som også går for varians.)

1.4.3 Feature engineering

Annvend lineær regresjonen til formen av testdataene dine! Gjør om featuresene x fra linear til polynom ved $x_1, x_2 = x, x^2$, og sett opp lineærregresjonen med to features istedet.
NB! funker for alle tilfeller, ikke kun for veiledet læring eller regresjon!

Generelt er feature engineering alle operasjoner vi utfører på datasettet vi bruker til maskinlæring, og inkluderer:

- **Feature selection**, altså utvelgelse av features, som da vi valgte å ikke ta med "Name" i klassifiseringsoppgaven på Titanic-dataene.
- **Feature preprocessing**, altså preprosessering av features, som da vi skalerte "Age" til intervallet (0, 1) for Titanic-dataene.
- **Feature extraction**, altså utvinning av features, som da vi nettopp laget en feature x_2 fra x. Det er også mulig å kombinere flere eksisterende features til nye.

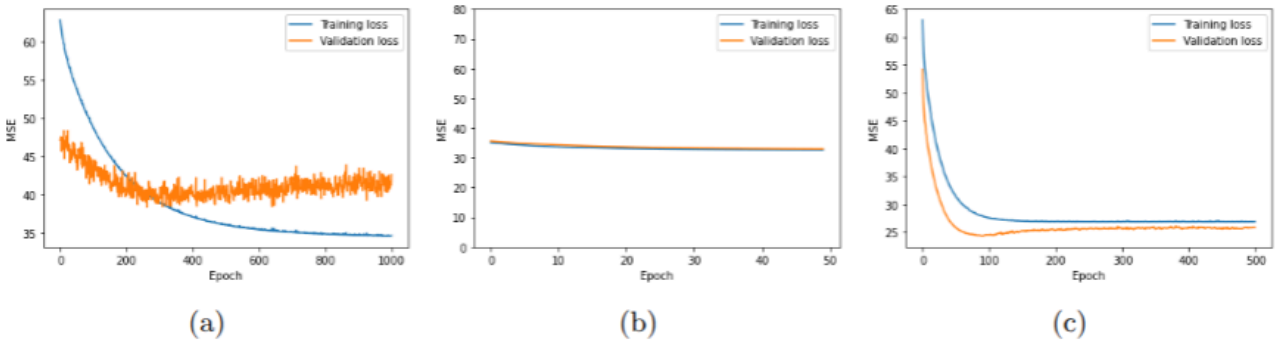
1.4.4 Trening, testing og validering

Overtilpasning skjer når modellen har kapasitet til å tilpasse seg så godt til treningsdataene at det går utover generaliseringsevnen på testdatene. Viktig å kunne detektere overtilpasning slik at vi kan juste hyperparametrene underveis i treningen. Men vi kan ikke bruke trenings- eller testdataene til å tilpasse hyperparametrene, så det er vanlig å dele datasettet i tre deler:

- **Treningsdata** brukes til å tilpasse modellparametrene (trening).
- **Testdata** brukes for å rapportere ytelsen til den endelige modellen, brukes ikke til å gjøre noe som helst tilpasning.

- **Valideringsdata** brukes for å monitorere modellen under trening, for å kunne justere hyperparametrene.

For å ta i bruk valideringsdataen, må treningsprosedyren regne ut relevante metrikker på valideringsdataene et gitt antall ganger i løpet av treningen. Dette kan gjøre f.eks med å beregne tapet for både trenings- og valideringsdata og plotte opp mot hverandre:



a) Overtilpasser, b) undertilpasser, c) optimal tap for test og validering

- Hvis tap på treningsdataene synker jevnt, mens loss pp valideringsdataene er høyere og/eller flater ut, overtilpasser modellen.
- Hvis begge kurvene flater ut med høy loss, undertilpasser modellen. Det betyr som oftest at den ikke har kapasitet eller riktig form til å tilpasse seg til dataene. Det mest åpenbare tegnet på undertilpasning er høy loss på treningsdataene.
- Hvis begge kurvene synker jevnt og flater ut, betyr det at modellen klarer å modellere trenings- dataene, og samtidig klarer å generalisere til nye data.
  Kan også bruke metrikkene til å justerehyperparametre, som læringsraten eller avgjøre hvor mange epoker en skal bruke for treningen.

1.4.5 Kryssvalidering

Hvordan skal dele inn i trenings-, test- og valideringsdata? Skal gjøres tilfeldig, men det vil påvirke testresultatet, avhengig av datasplitten. En løsning er #kryssvalidering.
Enkleste formen er **leave-on-out-cross-validation** (#LOOCV), hvor en observasjon fjernes fra det opprinnelige datasettet og brukes til testing.
For totalt n datapunt vil vi kjøre trening på $n-1$ datapunkt, også loope igjennom for hvert datapunkt og bruke en og en som testdataen. De n resultatene tar vi gjennomsnittet av for å få LOOCV-estimatet.

Annen form er #k-fold_cross_validation, med k-splitter istedet for n. Man kjører da samme oppsett, men med $n-k$ treningsdatapunkt og k testdatapunkt.

1.4.6 Regresjonstrær

For tydelige fordelinger av target data med ukjent funksjonsform vil ren regresjon funke dårlig. Istedet kan fordele dataene i flere regioner ved å bruke beslutningstrær, hvor hver splitt i treet deler opp datarommet. Deretter kan en predikere en verdi per område.

Table 7: Forskjeller mellom beslutningstrær og regresjonsmodeller.

Trær	Regresjon
Feature scaling er ikke nødvendig. Beslutningstrær deler inn dataene i regioner, så det er ikke nødvendig å skalere dem.	Features bør ha samme størrelsesorden, helst $\mathcal{O}(1)$, så ikke tilpasningen av modellparametrene fører til store gradienter.
Hvis læringsalgoritmen ikke har begrensninger i tredybde, kan treet bli for dypt og overtilpasse.	Funksjonsformen er bestemt. Læringsalgoritmen kan ikke legge til ledd og lage en mer kompleks modell, eller fjerne ledd om mindre kompleksitet er tilstrekkelig.
Har en overordnet struktur, men den spesifikke oppbygningen – *arkitekturen* – bestemmes under trening.	Har en gitt likningsform med antall ledd, og parametrene til hver variabel og variabelkombinasjon tilpasses.

En viktig egenskap for regresjonsmodeller og beslutningstrær er at de er tolkbare:

- Fore regresjonsmodeller, om x_i er stor, skjønner vi at x_i har stor betydning på prediksjonen
- For beslutningstrær er splittkriteriene forståelig for mennesker, og features som splittes tidlig er viktigere enn de som splittes senere.

1.5 Ensemble-modeller

Alle modeller har sine antakelser og svakheter. Tanken bak #ensemble_learning er at flere modeller kan kombineres, slik at de kompenserer for hverandres svakheter, og til sammen utgjør en samling (ensemble), som benytter seg av hver enkelt modells styrke.

Vi har konseptuelt 3 ulike måter for å kombinere flere modeller:

- **Parallelt:** Flere modeller trenes uavhengig av hverandre, og prediksjonene deres kombineres til en enkelt prediksjon.
- **Sekvensiet:** Flere modeller kommer etter hverandre, og hver modell opphever feilen begått av foregående modell. Tail sammen kommer rekken av modeller frem til en prediksjon, der hver modell har minimert feilen gjort av modellen før.
- **Hierarkisk:** Vi bruker en (eller flere) modeller til å kombinere prediksjonen fra en foregående parallellkombinasjon av flere modeller.

Når vi bruker en trent modell til å gjøre prediksjoner, sier vi at vi gjør #inferens. Skillet mellom trening og inferens er viktig, særlig synlig i ensemble learning.

Kan lage et parallellt ensemble på flere måter:

- Trene samme type modell med n ulike valg av hyperparametre, for hvert datapunkt vil vi få n ulike prediksjoner som vi **aggregerer** til en endelig prediksjon.
- Trene n ulike type modeller, og aggregere prediksjonene til en endelig prediksjon.
- Kan også trene ulike modellene med ulike deler av treningsdataene.

1.5.1 Bagging

Må ta stilling til to spørsmål ved bruk av ensemble med ulike deler av treningsdataene og kombinere prediksjonene:

1. Hvordan aggregere de ulike modellenes prediksjoner?
2. Hvordan velge ut hvilke deler av treningsdataene hver enkelt modell trener på?

Aggregering:

For regresjon tar en som oftest gjennomsnitt av alle modellenes prediksjoner.
For klassifisering gjøres det som oftest med å velge flertallet av enkeltmodellenes prediksjoner eller å gjøre en vektet avstemning mellom modellene.

Utvalg av treningsdata:

#bootstrapping er et sentralt konsept i utvalg av treningsdata. en underliggende tanken bak bootstrapping er at vi vet at vi ikke kan samle nok data til å representere den underliggende fordelingen bak et fenomen perfekt, men gitt et stort nok datasett kan vi få til et tilstrekkelig representativt utvalg. Likevel vil et representativt utvalg ikke uten videre fortelle oss om usikkerheten i estimatene vi gjør basert på disse dataene, altså hvor stor spredning det har. Gitt datasettet vi har samlet kan vi dog lage et estimat av spredning, eller usikkerhet, ved hjelp av teknikken **bootstrapping**.
Dette går ut på å trekke flere datapunkter fra det samme datasettet med tilbake-legging (dette er viktig: det samme datapunktet kan finnes flere ganger i resulterende datasett), og slik ende opp med flere ulike datasett fra det ene datasettet vi startet med. Vi bruker disse ulike datasettene til å estimere den samme størrelsen flere ganger, og slik ende opp med en fordeling av estimatene.

Denne fordelingen kan vi bruke til å beregne en spredning, eller usikkerhet, i estimatet vårt. Vi startet altså med ett datasett som vi kunne lage ett estimat fra, men har ved hjelp av bootstrapping skaffet oss en fordeling – uten å ha fått tilgang til flere datapunkter eller datasett. Vi har laget mer uten å måtte samle mer data, altså “pulled us up by our own bootstraps”.

[BAGGING kommer fra Bootstrap + AGGregerING]

Eksempel på ensemblemodell som bruker bagging er #random_forest. Lages ved å sette sammen ulike beslutningstrær, eventuelt stumper. For å få god modell må de ulike trærne være diverse og uavhengige, som gjøres ved å trene på ulike deler av dataene med bootstrapp-teknikken, bruke ulike utvalg data-features og til slutt agreggere prediksjonene sammen - bagging har skjedd.

1.5.2 Boosting

Brukes ofte innen ML for algoritmer som iterativt (sekvensielt) trener svake modeller på en datafordeling, og kombineres til en sterk modell (ensemblet). Går ut på at feilene

begått av en modell gjør den påfølgende modellen i iterasjonen bedre, derved "booster" den. Vi ser på to algoritmer som gjør dette:

AdaBoost-algoritmen bygger et ensemble av modeller som korrigerer hverandres feil gjennom en iterativ treningsprosedyre. I starten av prosedyren har alle punktene i treningsdataene samme vekt. Etter hver modell trenes, ser vi basert på targets hvilke datapunkter som modellen har størst tap, og øker vektene for disse datapunktene.

Gradient-boosting baseres ikke på vekting av observasjoner. Hver modell predikerer forskjellen mellom targets og den forrige modellens prediksjon, såkalte #pseudo_residuals. Det nye ensamblet lages ved å følge læringsregelen

```
new_ensemble = previous_ensemble - learning_rate * new_tree
```

Minner om gradient descent, men gjør gradient descent i rommet over alle mulige trær ensemblet kan bestå av (istedet for i rommet over alle mulige parameterverdier).

Det er en god regel å alltid bruke en ensemble-modell som referanseverdi for hvor godt en modell kan gjøre det, når du jobber med et maskinlæringsproblem med tabulære data.

2 Nevrale nettverk

Vi skal nå se på å lage ikke-lineære modeller som transformerer dataene.

2.1 Perceptron

Perseptronet er forgjengeren til moderne nevrale nettverk, introdusert av McCulloch og Pitt i 1943 og bygget første implementasjon av Rosenblatt i 1957. Perseptronet skulle være en maskin; ikke et program, som besto av en eller flere beregningsenheter/noder som ble opprinnelig kalt #threshold_logic_units (**TLU**). Tokk en input x og mappet til en output $f(x)$ som gir ut en binær verdi

$$f(\mathbf{x}) = (\mathbf{wx} + b)$$

Der er Heavside stegfunksjonen (x) step-funksjonen) som vi gjerne kaller for en #aktiveringsfunksjon. Kan skrives med kode som:

```
activation = sum(weigh_i * x_i) + bias

prediction = 1 if activation >= 0 else 0
```

Brukes likt som tidligere med klassifisering siden binær funksjon. Men kommer problem med når vi skal derivere tapsfunksjonen fordi ikke kan derivere Heavside funksjonen. I opprinnelige formulieringen av perseptronet er leddet med den deriverte utelatt og bruker heller

$$w_i \leftarrow w_i + \eta(y_i - \hat{y}_i)$$

Vanlig i kode å skrive om θ eller b inn i $\mathbf{w}$ og $\mathbf{x}$ slik at første element i $\mathbf{x}$ er 1 og $\mathbf{w}$ har et tilsvarende ledd for θ eller b, også skrive:

$$\mathbf{w} = \mathbf{w} + \texttt{learning_rate} * (\texttt{expected} - \texttt{predicted}) * \mathbf{x}$$

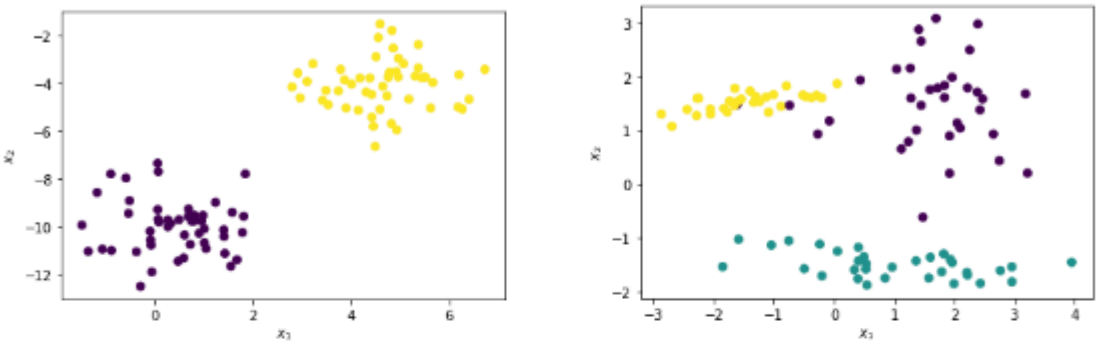
Kan bruke med `sklearn.linear_model.Perceptron`:

```python
perceptron = Perceptron().fit(X_train, y_train)
y_pred = perceptron.predict(X_test)
```

Til info er dette en wrapper rundt `SGDClassifier` som vi har brukt tidligere, med `loss="perceptron"` og `learning_rate="constant"`. Vi finner de tilpassede parameterverdiene tilsvarende som for lineær regresjon:

```
ws = perceptron.coef_
bs = perceptron.intercept_
```

Se på dataene i figur 17b. **Oppgave:** Hvor mange features og klasser har dataene? Hvor mange vekter trenger et perseptron for å tilpasse dem? Det kan være nyttig å tegne en figur.



(a) (b)

Figure 17: Klassifiseringsdatasett med (a) to klasser, generert av `sklearn.datasets.make_blobs`, og (b) tre klasser, generert av `sklearn.datasets.make_classification`.

For å gjøre denne klassifiseringsoppgaven, kan vi gjenbruke koden fra tidligere, og treningsprosedyren forblir den samme. Hvis man har implementert `.fit()` og `.predict()` selv, må koden eventuelt tilpasses for å håndtere en vektmatrise (i stedet for en -vektor), da parameteroppdateringen nå følger:

$$w_{i,j} \leftarrow w_{i,j} + \eta\,(y_j - \hat{y}_j)\,x_i\,. \tag{76}$$

Her er

- $w_{i,j}$ vekten på forbindelsen mellom input i og node j,
- η læringsraten,
- $\hat{y}_j$ og y_j henholdsvis output og target for klasse j,
- x_i feature-verdi i for det aktuelle datapunktet.

Vi gjenbruker koden fra tidligere, og kan finne ut hvor mange klasser og features vi har (som vi strengt tatt bør ha kontroll på før vi tilpasser modellen), og hvor mange vekter modellen har, ved hjelp av følgende kode.

```
perceptron = Perceptron().fit(X_train, y_train)
ws = perceptron.coef_
bs = perceptron.intercept_
print("Features:", X.shape[1])
print("Classes:", len(list(set(y))))
print("Weights:", ws.shape)
print("Biases:", bs.shape)
```

Har vi to features og tre klasser, må modellen ha to input-noder med forbindelser til tre output-noder. Da får vi seks vekter i **w** og tre bias-verdier i **b**.

Logiske operasjoner

Perseptroner kan brukes til å gjøre logiske operasjoner. Husk formelen for f(x):

- NOT: en variabel x_1 med $(w_1, b) = (-1, 0)$
- AND: to variable x_1, x_2 med $(w_1 w_2 b) = (1, 1, -1)$
- OR: to variable x_1, x_2 med $(w_1, w_2 b) = (1, 1, -0)$
- XOR: Flere TLU, run an OR and a NOT AND, take the outputs and run through AND

2.2 Aktiveringsfunksjoner

Ulempe med perseptronet er bruken av Heavside funksjonen gjør den veldig sensitiv til små endring i input. I moderne nevrale nettverk er mer vanlig å bruke #Sigmoid-funksjonen, #softmax, og #ReLU. Disse brukes til ulike formål.

Modell	Brukes til	Funksjon .
Sigmoid	Binær klassifisering	$\sigma(x) = \frac{1}{1+e^{-x}}$
softmax	Ikke-binær klassifisering	$\text{softmax}(\mathbf{x})_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$
ReLU	For MLP	$\text{ReLU} = \max(0, x)$

Softmax

Vanlig å bruke softmax i output-nodene. Sikrer at aktiveringene summer til 1. DA representerer hver av output-nodene datapunktets predikerte tilhørighet til de respektive klassene, og må ha en output-node per klasse.

ReLU

Rectified linear unit (#ReLU) brukes i moderne nevrale nettverk som har som regel flere lag, mellom input- og output-lagene. (Se 2.3) Slike modeller kalles multi layer perceptrons (#MLP). Vi velger fritt aktiveringsfunksjoner til lagenes noder selv, faget holder seg til ReLU.

MLP-modeller som ikke gjør klassifisering men regresjon, må ha output-noder som kan returnere kontinuerlige tallverdier som ikke er begrenset til et intervall (som [0, 1] i klassifisering). Som regel brukes da en lineær aktiveringsfunksjon i output-laget

2.3 Arkitektur

Vi kan sette sammen noder på ulike måter for å lage nevrale nettverk. Måten de er satt sammen kalles *netterkets arkitektur*. Nevrale nettverk består av lag av noder stablet i høyden. Det vil si noder som ikke mottar input fra hverandre. Informasjon går kun fra noder i nabo lag, ikke mellom noder i samme lag. Nevrale nettverk består av 3 typer lag:

- **Input-laget** er det første laget. Mottar og sender data inn i nettverket. Samme antall noder som dataene har features.
- **Output-laget** er det siste laget. Representerer nevrale nettverkets prediksjon. Om veildet læring må det har samme dimensjonalitet/antall noder som targets i dataene.

- **Indre lag** er alle lagene mellom I/O lagene. Blir og kalt skjulte lag.

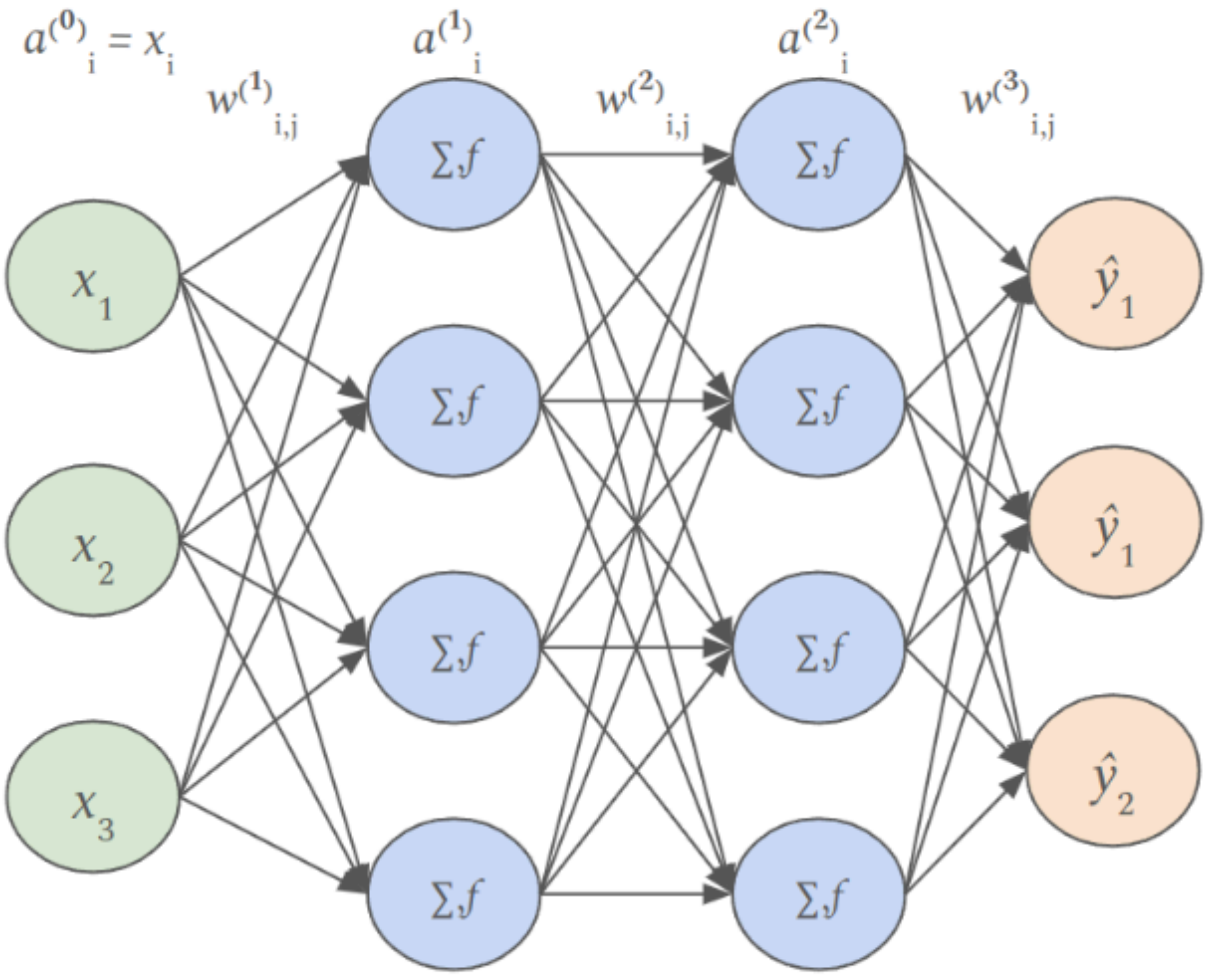


Figure 19: Skisse av et nevralt nettverk, som vist i forelesning.

Ulike arkitekturer er bedre og dårligere egnet tl ulike oppgaver. Eks brukes konvolusjonslag til bildegjenkjenning og transformer-blokker til sekvensiell data som språk. Skal ikke se på i faget her, bare **arkitekturer der alle nodene i hvert lag er koblet til alle nodene i nabo lagene.** Kalles MLP eller **fully connected feed-forward** nettverk. Generelt har vi følgende uttrykk for aktiveringen a til en gitt node med indeks j i lag ℓ av det nevrale nettverket

$$a_j^\ell = g\left(\sum_{i=0}^{n} w_{ij}^\ell a_i^{\ell-1}\right) \quad |a_i^{(0)} = x_i$$

Her er

- ℓ indeks for lag, indeks 0 er input
- i indeks for node i forrige lag
- j indeks for node i det aktuelle laget
- $a_i^{\ell-1}$ aktiveringen av node i forrige lag $\ell - 1$
- g aktiveringsfunksjonen i det aktuelle laget.
  Eksempel: Første noden i andre laget i bildet:

$$a_1^{(1)} = g\left(\sum_{i=0}^{n} w_{i1}^1 x_i\right)$$

Vi tar for oss tilfellet der vi to input-features, og har bygget et skjult lag bestående av tre noder. Nettverkets videre arkitektur er uten betydning for den aktuelle diskusjonen. Vi har altså $(i = 1, 2)$, og $(j = 1, 2, 3)$. Vi kan skrive aktiveringene i nettverkets første indre lag som

$$a_1 = g\left(w_{11}^{(1)} x_1 + w_{21}^{(1)} x_2\right) \tag{82}$$

$$a_2 = g\left(w_{12}^{(1)} x_1 + w_{22}^{(1)} x_2\right) \tag{83}$$

$$a_3 = g\left(w_{13}^{(1)} x_1 + w_{23}^{(1)} x_2\right) \tag{84}$$

etter å ha skrevet ut summen over input-laget. De tre likningene over kan skrives på matriseform som følger

$$\begin{bmatrix} a_1^{(1)} \\ a_2^{(1)} \\ a_3^{(1)} \end{bmatrix} = g\left(\begin{bmatrix} w_{11}^{(1)} & w_{21}^{(1)} \\ w_{12}^{(1)} & w_{22}^{(1)} \\ w_{13}^{(1)} & w_{23}^{(1)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right). \tag{85}$$

Kontroller at matrisemultiplikasjonen på høyre side skjer mellom to matriser med dimensjoner henholdsvis 3×2 og 2×1, hvilket resulterer i en matrise av dimensjon 3×1, som er det vi har på venstre side av likhetstegnet.

Generelt består det første laget i nettverket av n noder, og det første indre laget av m noder. Overgangen fra input-laget til det første indre laget innebærer altså en transformasjon fra n input features til m nye verdier. Dette gjelder for alle lagene i et nevralt nettverk: De gjør en transformasjon av dataene de mottar, til et nytt datarom av samme dimensjon som antall noder i det aktuelle laget. På grunn av ikke-lineariteten til aktiveringsfunksjonen g, er transformasjonen ikke-lineær. *Oppsummert gjør hvert lag i det nevrale nettverket en egen ikke-lineær transformasjon av dataene.* Dette kan vi tolke som en automatisk feature-transformasjon som utvikles i takt med at det nevrale nettverket lærer fra data.

2.4 Backpropagation

I stedet for gradient descent likt som med lineær regresjon, så på grunn av lagene i nevrale nettverk må en ta hensyn til at en ikke vet hva som er "riktige verdien" til de ulike nodene i indre/skjulte lag. Vi vil fremdeles bevege oss i parameterrommet, og estimerer dermed gradienten til tapsfunksjonen som funksjon av parametrene med tilbakepropageringe (#backpropagation).

Vi bruker et eksempel for å få en intuitiv forståelse, og igjen anbefales det at du tegner en figur, gjerne inspirert av slides brukt i forelesningen. Anta at vi har et nevralt nettverk med to input-noder, to indre lag med henholdsvis fire noder, og et output-lag med tre noder. Anta videre at det nevrale nettverket har gjort en prediksjon $\hat{y} = (0.1, 0.1, 0.8)$ på et datapunkt med target $y = (1, 0, 0)$. Vi ser at både $\hat{y}_1$ og $\hat{y}_3$ har store avvik fra target-verdiene sine, mens $\hat{y}_2$ har et lite avvik fra sin tilsvarende target-verdi. Vi bruker indeks $l = 0$ for input-laget, $l = 1, 2$ for de to indre lagene, og $l = 3$ for output-laget. Vi ser på aktiveringen til det første (øverste) nevronet i output-laget

$$\hat{y}_1 = a_1^{(3)} = g\left(w_{11}^{(3)} a_1^{(2)} + w_{21}^{(3)} a_2^{(2)} + w_{31}^{(3)} a_3^{(2)} + w_{41}^{(3)} a_4^{(2)}\right). \tag{102}$$

Her går altså indeks i i likning 96 over $i = (1, 2, 3, 4)$, mens $j = 1$, siden vi ser på den første noden, og $l = 3$, siden vi ser på det siste laget. I likningen over er det flere ting vi kan endre for å minke tapet: vi kan endre på parametrene b og $w_{i1}^{(3)}$ direkte, og aktiveringene av nodene i det tidligere laget, $a_i^{(2)}$, indirekte. Legg merke til at om vi endrer parametrene $w_{i1}^{(3)}$, må disse endres proporsjonalt med aktiveringene $a_i^{(2)}$. Vi ønsker å øke output $a_1^{(3)}$, som betyr at vi må styrke koblingen til aktive nevroner i lag 2, dvs nevroner med stor $a_i^{(2)}$. Enkelt sagt: Jo større a-ene er, jo større effekt har w-ene. Vi kan gjøre en tilsvarende analyse for $a_i^{(2)}$, for om alle $a_i^{(2)}$ som multipliseres med en positiv $w_{i1}^{(3)}$ ble sterkere, og alle $a_i^{(2)}$ som multipliseres med en negativ $w_{i1}^{(3)}$ ble svakere, hadde $a_1^{(3)}$ blitt større, som er det vi ønsker å oppnå. Enkelt sagt: Jo større w-ene er, jo større effekt har a-ene. Utfordringen er at vi ikke kan endre aktiveringene til indre lag direkte; vi kan kun endre dem indirekte gjennom parametrene deres. Vi må derfor endre på parametrene $w_{ij}^{(2)}$ tilhørende alle nodene i lag $l = 2$.

Før vi tar dette steget bakover i nettverket er det lurt å zoome ut, og huske på at vi kun har sett på én av verdiene i $\hat{y}$-vektoren: både $\hat{y}_2$ og $\hat{y}_3$ har sine egne preferanser for hvordan vektene og aktiveringene i de foregående lagene bør endres for at disse to delene av prediksjonen skal komme nærmere targets. Det er ikke gitt at $\hat{y}_1, \hat{y}_2$ og $\hat{y}_3$ er enige om hvilke endringer som bør gjøres, men vi samler uansett de ønskede endringene i lag $l = 2$ i en liste, før vi flytter oss et steg bakover i nettverket. Listen inneholder blant annet ønskede endringer for aktiveringene $a^{(2)}$, på samme måte som vi tidligere visste i hvilken retning vi ønsket å endre de ulike verdiene i $\hat{y}$, og basert på disse kan vi utføre samme analyse som vi gjorde under likning 102. Når vi har gjort dette for alle nodene i lag $l = 2$, lager vi en liste over ønskede endringer i lag $l = 1$, før vi tar et ytterligere steg tilbake i nettverket.

På dette tidspunktet har vi kommet til input-laget; det gir ikke mening å lage en liste over ønskede endringer i lag $l = 0$, siden dette er input-laget og aktiveringene svarer til verdiene i datasettet. Vi har derfor kommet til veis ende, ved å jobbe oss stegvis *bakover* i nettverket fra output, gjennom alle lagene, til input. Mer formelt har vi *propagert* feilen i prediksjonen *tilbake* til input, og gjennom denne prosessen funnet en liste over hvordan nettverkets parametre bør endre seg, altså *gradienten* vi trenger til parameteroppdateringen. Dette er essensen i backpropagation. Til slutt er det verdt å merke seg at dette må gjøres for alle datapunktene i treningsdatasettet; vi er ikke ute etter at modellen skal lære seg å predikere at $y = (1, 0, 0)$, men tilpasse prediksjonen til input-dataene. Akkurat som for lineær regresjon må vi altså sende alle treningsdataene (én epoch) gjennom nettverket, gjerne flere ganger (flere epochs).

2.5 Bygge, trene og predikere

Python biblitoker for å lage nevrale nettverk: `keras` & `tensorflow`.

Eksempel på nevral nettverk:
Kinda shit klassifisering

PYTHON

```python
import matplotlib
matplotlib.use("QtAgg")
import matplotlib.pyplot as plt
import numpy as np
import keras
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from keras import ops
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

x = ops.ones((10,2)) # Makes an input matrix with 3 rows and 10 coloums with
ones (1).

model = Sequential()
model.add(keras.Input(shape=(2,)))
model.add(Dense(6, activation="relu", name="layer1"))
model.add(Dense(7, activation="relu", name="layer2"))
model.add(Dense(1, activation="sigmoid", name="output"))

model(x)
model.compile(
        optimizer="SGD",
        loss="binary_crossentropy",
        metrics=["accuracy"]
        )


X, y = make_moons(n_samples=2000, shuffle=True, noise=0.2, random_state=42)

x_temp, x_test, y_temp, y_test = train_test_split(X, y, test_size=0.2)
x_train, x_val, y_train, y_val = train_test_split(x_temp, y_temp,
test_size=0.2)

history = model.fit(x_train, y_train, batch_size=64, epochs=10,
validation_data=(x_val, y_val))

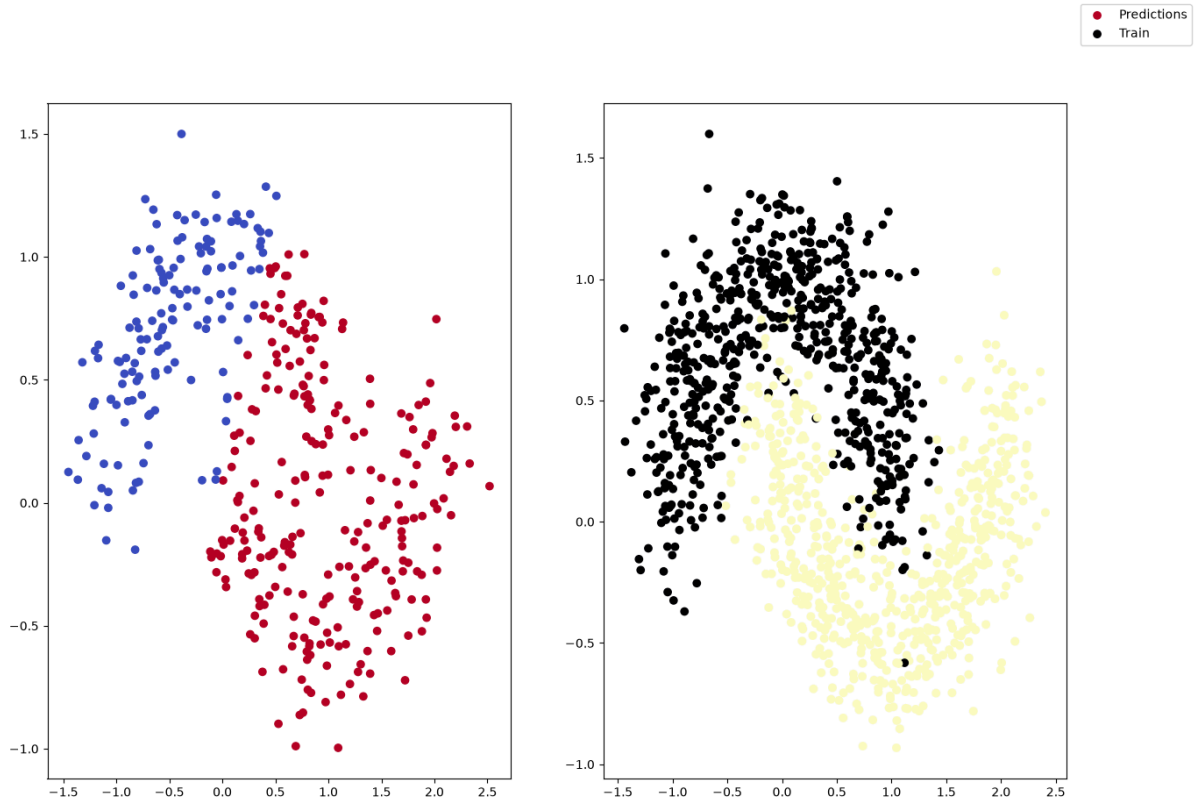
predictions = model.predict(x_test)
predictions = (predictions > 0.5).astype(int)

x_t, y_t = np.array(x_test).T
x_tr, y_tr = np.array(x_train).T

fig, (ax, xa) = plt.subplots(1,2, figsize=(15,10))
ax.scatter(x_t, y_t, c=predictions[:,0], cmap="coolwarm",
label="Predictions")
xa.scatter(x_tr, y_tr, c=y_train, cmap="magma", label="Train")
fig.legend()

#train_losses = history.history["loss"]
#val_losses = history.history["val_loss"]
#epochs = np.arange(1, len(train_losses)+1)
```

```
#plt.plot(epochs, train_losses, label="Train loss")
#plt.plot(epochs, val_losses, label="Validation loss")
#plt.xlabel("Epochs")
#plt.ylabel("Loss")
#plt.legend()
plt.show()
```



Prinsipielt jobber vi med nevrale nettverk som med andre maskinlæringsmodeller, men vi har en større frihet i valg av arkitektur og antall parametre. Det krever trening å lære seg hvilke arkitekturer som er egnet for hvilke læringsoppgaver og datastrukturer. Oppsummert koker treningsprosessen av nevrale nettverk ned til følgende steg:

- Modellen bygges, og kompileres med valg av optimizer, tapsfunksjon, early stopping, med mer.
  `model.compile()`
- Når modellen er bygget kan vi trene den.
  `model.fit()`
- Deretter kan vi bruke den til å predikere, altså gjøre inferens.
  `model.predict()`
- Når modellen er trent, evaluerer vi den på egnet metrikk.
  `model.evaluate()`
- Når vi går hjem for dagen kan vi lagre modellen med
  `model.save('sted/for/lagring/navn.keras')`
- ...og laste den opp igjen neste morgen.
  `model = keras.models.load_model(sted/for/lagring/navn.keras).`

3 Uveiledet Læring

Vi har nå ikke fasit på dataene våre, vi kaller det for #unlabeled_data, det vil si data uten targets. Handler om å gjøre maskinlæring med læringsalgoritmer som finner mønstre i data uten targets. VI skal se på tre kategorier uveiledet læring:

- Clustering / klynging
- Dimensionality reduction / dimensjonsreduksjon
- Outlier detection / anomalideteksjon

3.1 Clustering

Kan tolkes som en form for "klassifisering uten labels", med formålet om å finne egnede grupperinger av like datapunkter i et datasett. Kan bruke flere likhetsmetrikk som avstand mellom datapunktene. Vi skal se på to ulike læringssignal, aka tap å minimere for optimaliseringsprosessen:

- K-means, likhet defineres som avstand.
- DBSCAN, likhet defineres som tetthet.

3.1.1 k-means

k-means deler datasettet bestående av n datapunkter, inn i k klynger. Hvert datapunkt tilordnes klyngen hvis klyngesentrum er nærmest datapunktet.

1. Velg et tall k, som representerer antall klynger.
2. Velg k tilfeldige klyngesentroider, dvs midten av klyngene og må ikke være et datapunkt selvom kan være.
3. Tildel hvert datapunkt en tilfeldig klynge.
4. Beregn den Euklidske avstanden mellom hvert datapunkt og alle sentroidene.
5. Tildel hvert datapunkt den nærmeste sentroiden fra steg 4.
6. Velg nye sentroider ved å regne ut middelverdien (`npmean`) for hver klynge.
7. Gjenta steg 4, 5 og 6 intill klyngen ikke endrer seg.

Utfordring er å velge gode tilfeldige sentroider, og sannsynligheten for å velge uegnede steder øker eksponensielt med k.
Intuitivt: Vi har k valg per tilfeldig plassering, altså k^k like sannsynlige sekvenser av sentrioder.
Sekvensene vi er ute etter er de hvor alle k sentroidene svarer til ulike labels, altså én sentroide per klynge. Dette er bare ulike ordninger av k klynger, og vi har til sammen $k!$ av disse. Antall gunstige over mulige er derfor $k!k^k$.

Stirlings approksimasjon lar oss skrive om fakultet som

$$k! \approx \sqrt{2\pi k}\, e^{-k} k^k$$

Som vil si eksponensielt avtakende med for å initialisere k sentroider i unike klynger.

En annen vanlig løsning er å velge første sentroiden tilfeldig, også plassere neste sentroide på datapunktet som er lengst borte fra den første, gjentatt for alle k sentroidene.

kmeans+ +

Til slutt skal vi se på `kmeans++`, som bruker en mer avansert måte å velge plasseringene til sentroidene. I stedet for å velge sentroide j på datapunktet lengst borte fra de forrige sentroidene, velges datapunktet med en viss sannsynlighet, der sannsynligheten er proporsjonal med kvadrert avstand fra den nærmeste initialiserte sentroiden. Grunnen til at dette er lurt, er at strategien der datapunktet lengst borte fra forrige sentroide velges som sentroide j, er at det som oftest vil plassere sentroiden i utkanten av en klynge. Med sannsynlighetstilnærmingen velges fremdeles et datapunkt som er langt borte fra forrige klynge – proporsjonal med kvadrert avstand – men med en viss tilfeldighet. Denne tilfeldigheten øker sannsynligheten for å havne nær midten av den faktiske klyngen. Det finnes et bevis på at denne fremgangsmåten for plassering av sentroider er forventet å være suboptimal med maksimalt en faktor $\log(k)$.

Vi kan enkelt lage en k-means-modell ved hjelp av python-biblioteker som keras. Jeg abefaler å bruke `keras.cluster.MiniBatchKmeans`, særlig for store datasett. I stedet for å bruke hele datasettet hver iterasjon, bruker en den liten tilfeldig trekning (en mini batch). Vi kan angi k gjennom argumentet `n_clusters`, og velge initialiseringsmetode for klyngesentrene gjennom argumentet `init`. Hvis vi ikke oppgir en verdi for k, vil algoritmen selv prøve å velge en optimal verdi for k. Dette kan fungere godt, men vil vel så ofte fungere mindre godt. *Valg av k er blant de store utfordringene innen clustering*, og det finnes flere metrikker for å måle kvaliteten på clusteringen. Biblioteket `sklearn.metrics.cluster` har flere metrikker innebygget, og dere bør ha hørt om tre av dem:

- **Inertia**: Sum av kvadrert avstand mellom punkter og sentroide. Lav verdi betyr kompakte klynger, som er bra. Svakheten ved denne metoden er at den letteste måten å oppnå en lav verdi, er ved å plassere en sentroide på hvert datapunkt, altså ha størst mulig k.
- **Silhouette Coefficient**: $\frac{b-a}{\max(a,b)}$, hvor a angir midlere avstand innad i klyngen, såkalt mean intra-cluster distance, og b angir midlere avstand til nærmeste klynge, såkalt mean nearest-cluster distance. Den måler altså hvor nært et datapunkt er til sin egen klynge, sammenliknet med andre klynger. Den beste verdien til denne metrikken er 1, som betyr at punktet er nærmere sin egen enn andre klynger. 0 betyr at klyngene overlapper eller at punktet er på randen mellom to klynger, og den dårligeste verdien er -1, som representerer at datapunktet er i feil klynge.
- **Calinski Harabasz score**: Ratio mellom sum av kvadrert avstand mellom klynger, hvor en høy verdi er bra, og sum av kvadrert avstand mellom punkter i samme klynge, hvor en lav verdi er bra. Den måler altså hvor langt det er mellom klynger normalisert til hvor store klyngene er. En høy verdi av denne metrikken representerer tette klynger med god separasjon.

Med utgangspunkt i et datasett uten labels og hvor vi ikke klarer å se hvilken verdi av k vi bør velge, kan det være nyttig å tilpasse flere k-means-modeller med ulike hyperparametre, og regne ut flere metrikker for å se om vi klarer å identifisere optimale verdier.

Som vi ser eksempler på i forelesningene, finnes det tilfeller der k-means ikke klarer å identifisere klynger selv om vi oppgir korrekt verdi av k. Dette skjer typisk når avstanden innad i klyngen er større enn mellom naboklynger. Dette er en konsekvens av at k-means bruker avstand som et mål på likhet mellom punkter. Dette peker på en viktig detalj, som gjelder i all maskinlæring: *Alle læringsalgoritmer har implisitte antakelser om problemet de skal løse.*

Generelt har k-means følgende ulemper:

- Følsom for antall sentroider valgt, automatisk valg fungerer ofte dårlig, og manuelt valg er kun så godt som utvalgsmetoden.
- Svak med outliers, siden disse kan trekke sentroidene vekk fra de faktiske klyngene.

- Fungerer dårlig i høydimensjonale rom fordi den baserer seg på Euklidsk avstand, som konvergerer til en konstant for store d.
  Og følgende fordeler:
- Lett å forstå og implementere
- Fungerer godt på store datasett (store n)

3.1.2 DBSCAN

Baserer seg på identifisere tre typer punkter i dataene:

1. Kjernepunkter: datapunkter som ligger nært midten av en klynge.
2. Ikke-kjernepunkter: Data punkter som tilhører klyngen, men ligger i utkanten.
3. Outliers: Datapunkter som ikke tilhører noen klynger. *En hovedstyrke ved metoden at den ikke prøver å putte disse punktene i klynger!*

Relevante hyperparametre for DBSCAN er ϵ, som angir maksimal avstand mellom to nabopunkter som kan tilhøre samme klynge, og `min_samples`, som angir minimum antall naboer et punkt må ha for å telle som et kjernepunkt. DBSCAN-algoritmen bruker følgende prosedyre:

1. Identifiser kjernepunkter ved å telle antall naboer innenfor en avstand ϵ. Hvis dette antallet er større enn `min_samples`, er datapunktene kjernepunkter.
2. Gå gjennom kjernepunktene og spre klyngen til datapunkter i nærheten. Hvis punktet er et kjernepunkt, bruk det til å utvide klyngen. Hvis ikke er datapunktet et ikke-kjernepunkt.
3. Identifiser alle punkter som ikke tilordnes en klynge, som outliers.

Vi vet altså om et datapunkt er en et kjernepunkt basert på hyperparametrene, allerede i steg 1. Etter denne prosedyren vet vi også forskjellen mellom outliers og ikke-kjernepunkt. Ikke-kjernepunkter har færre enn `min_samples` naboer innenfor en avstand ϵ, men de er innenfor en asvstand ϵ av et kjernepunkt. De er da med i klyngen, men bidrar ikke til å utvide klyngen.

Fordeler ved DBSCAN er at vi ikke trenger å spesifisere antall klynger, at algoritmen heller ikke gjør en antakelse om dette antallet, og at algoritmen har en metode for å identifisere outliers og dermed er robust for outliers. Ulempene er følsomhet for valg av ϵ og `min_samples`, og dessuten at disse er globale variabler: det kan hende at vi får datasett der klyngene har ulik tetthet for ulike datapunkter. Siden vi bare kan velge én verdi av henholdsvis ϵ og `min_samples`, er DBSCAN mindre egnet i tilfeller der klyngetettheten varierer. DBSCAN er også tregere for store datasett (stor n), fordi den må beregne avstander mellom samtlige punkter for å finne naboer og identifisere klyngepunkter, og er svakere i høydimensjonale data, fordi den baserer seg på et mål av avstand, som er utfordrende i høydimensjonale rom, på grunn av dimensjonsforbannelsen.

3.1.3 Hovedkategorier

Vi kan dele clustering-algoritmer i 4 hovedkategorier:

- **Sentroidebasert** Starter med sentroider, assosierer datapunkter med disse og tilpasser frem til konvergens.
- **Tetthetsbasert** Klynger defineres basert på tettheten til datapunkter. Punkter som ligger nærme hverandre defineres som klynger, men isolerte punkter defineres som outliers.
- **Fordelingsbasert** Gjør antakelsen at hver klynge består av punkter fra en sannsynlighetsfordeling, vanligvis en Gauss. Målet er å finne parametrene til denne fordelingen.
- **Hierarkibasert** Lager et hierarki av klynger. I starten er alle datapunktene en egen klynge, og på slutten tilhører alle datapunktene samme klynge. Klyngene som dannes underveis sorteres i et hierarki, og man velger ut de genererte klyngene fra et steg i hierarkiet.

3.1.4 Dimensjonsforbannelsen igjen

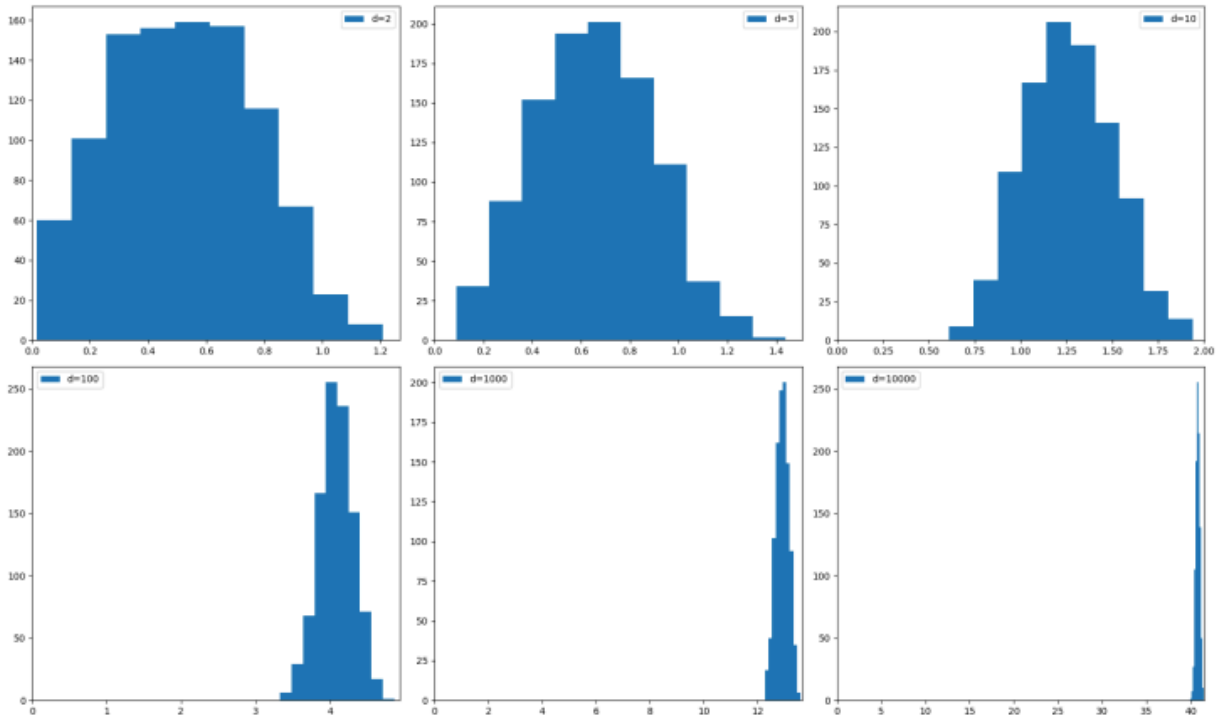


Figure 22: Fordelingen av Euklidske distanser mellom to punkter i en d-dimensjonal enhetskube.

Det finnes flere liknende øvelser vi kan gjøre for å få intuisjon. I slides fra forelesningen vises hvor stor andel av en d-dimensjonal enhetskube eller -kule som ligger i et tynt ytre skall av tykkelse ϵ. Basert på disse enkle øvelsene forstår vi at avstander skalerer eksponensielt med d. Dette gir opphav til det såkalte *Concentration of Distances*-fenomenet, hvis direkte konsekvens er at *relative* avstander blir mindre i høye dimensjoner. Avstandene i seg selv blir veldig store, slik at de relative *forskjellene* mellom avstander blir mindre. Dette vises tydelig i figur 22: fordelingenes middelverdier blir store med økende d, mens spredningene blir små sammenliknet med middelverdiene. Tidligere, i diskusjonen om veiledet læring, forstod vi at dette betyr at mesteparten av datarommet er tomt når vi samler inn data med flere features enn $\sim \mathcal{O}(10)$. I den nåværende diskusjonen forstår vi at dette har konsekvenser også for klynger i høydimensjonale rom: datapunkter i samme klynge bør være nærmere hverandre enn punkter i andre klynger. Siden datapunkter i høydimensjonale rom havner i $\sim$ samme avstand fra hverandre (Concentration of distances), klarer clustering-algoritmer generelt ikke å tildele datapunkter til riktig klynge. Merk at å same flere datapunkter *ikke* løser dette, siden problemet har opphav i hvordan avstander oppfører seg i høydimensjonale rom; ikke størrelse på datasett. For å hanskes med dette problemet vil vi derfor i neste omgang se på teknikker vi kan bruke for å redusere dimensjonaliteten i data, fortrinnsvis gjennom å beholde kun retninger (dimensjoner) som representerer strukturen i dataene våre.

3.2 Dimensjonsreduksjon

PCA og t-SNE sammenliknet

- **Mål:** PCA Bevarer variansen i dataene; t-SNE minimerer divergensen mellom to sannsynlighetsfordelinger av dataene.
- **Gjenbrukbarhet:** PCA lager nye basisvektorer, og kan derfor brukes igjen etter `.fit`; t-SNE er ikke-parametrisk, må tilpasses og brukes samtidig, og kan derfor ikke gjenbrukes.
- **Feature-generering:** PCA kan brukes til å lage nye features, tilsvarende eller basert på hovedkomponentene; t-SNE kan ikke brukes til å lage nye features, siden den ikke lager en matematisk funksjon som kan brukes på nye datapunkter.
- **Linearitet:** PCA trenger lineært separerbare data; t-SNE har ingen antakelse om linearitet.
- **Compute:** PCA er rask og lite beregningstung, mens t-SNE må tilpasse mange sannsynlighetsfordelinger og derfor er treg på store datasett.

3.3 Anomalideteksjon

Oppsummering anomalideteksjon

Vi kan oppsummere fremgangsmåten for anomalideteksjon med de tre metodene som følger:

- **k-means**: Lager k klynger av dataene, basert på avstand mellom klyngesentrum og datapunktene. Anomalier er datapunkter som ligger langt unna alle klyngesentrene.
- **DBSCAN**: Lager klynger av punkter med høy tetthet, og identifiserer anomalier som isolerte punkter, altså outliers. DBSCAN finner outliers som en del av modelleringen.
- **Isolation Forest**: Bygger trær der alle punktene havner i en egen løvnode. Anomalier er punkter som har kort gjennomsnittlig avstand fra rotnoden, altså er lettere å isolere enn resten av datapunktene.

3.4 Self-supervised learning???

4 Reinforcement learning

Mye teori bak #reinforcement_learning forkortet RL, men skal kun se på en del av den. Finnes mange læringsregimer, vi skal se på #Q-learning.

4.1 Markov Decision Process

NB! Agent her er ikke samme som Agent i dagens språkmodeller som Copilot og ChatGPT!

RL er en type maskinlæring hvor en #agent lærer å ta beslutninger i et miljø, omtalt som #environment, ved å utføre handlinger, omtalt som #actions, for å maksimere forventet belønning, omtalt som #expected_reward. Dette gjøres i en loop som Figur 29 under:'

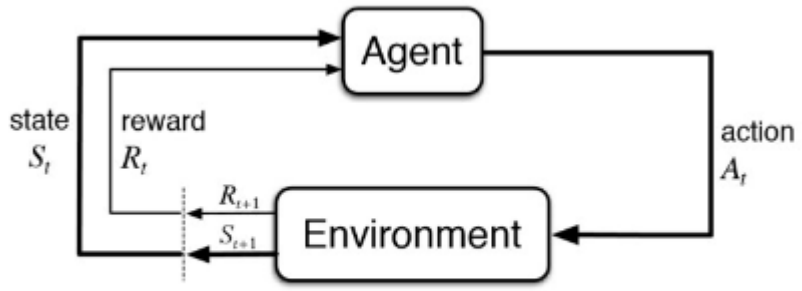


Figure 29: Reinforcement learning loop.

Miljøet gir agenten en state/tilstand S_t ved steg t som agenten velger en action/handling A_t basert på. Dette gir en reward/belønning R_{t+1} fra miljøet, og setter miljøet i en ny tilstand S_{t+1}. Og sånn fortsetter den i en loop.

Antar at den fremtidige tilstanden til miljøet kun er avhengig av inneværende tilstand og valgt handling; ikke tidligere states/actions. Dette tilsvarer en #Markov-prosess som vil si at den er uavhengig av historien. Blir også kallt memoryless. *"Fremtiden er uavhengig av fortiden gitt nåtiden"*.

En beslutningsprosess som har Markov-egenskapen, kalles en #Markov_decision_process (MDP). Formelt er en MDP en 4-tuple av

- S er set av states (State space)
- A er set av actions (Action space). Alternativt: A_S er settet av actions tilgjengelig fra state S
- $_A(S_t, S_{t+1}) = (S_{t+1}|S_{t,A_t})$ er sannsynligheten for at action A_t i state S_t vil føre til S_{t+1}
- $R_A(S_t, S_{t+1})$ er den umiddelbare rewarden for å transisjonere fra state S_t til S_{t+1} gjennom action A_t.

Vi skal ikke se på transisjonssannsynligheter, bare holde oss til deterministiske miljøer! Basert på en **læringsprosess i miljøet** som gir agenten states og rewards, skal agenten finne måte å velge ut beste action basert på en gitt state. Denne måten kalles en #policy, og vi bruker notasjonen $(A_t|S_t)$ for å anngi en policy for å velge action basert på state. Målet til en RL-agent er å finne en $optima policy$, som maksimerer total reward, omtalt som kumulativ reward, fra miljøet.

Agenten kan være mye forskjellig, og vi skal først se på en tabell og deretter et nevralt nettverk som agent. For at denne skal kunne lære en optimal policy, må den trenes på å velge action som gir høy reward, og vi må ha en måte å beregne kumulativ reward. Dette gjør vi gjennom en utility eller value function, som angir hvor verdifull en tilstand og action er på lang sikt, under en viss policy. Denne verdifunksjonen kvantiserer altså forventet verdi av å være i en gitt tilstand, utføre en gitt action, følge en gitt policy, og fortsette å følge denne policyen fremover. Agenten trenger verdifunksjonen for å kvantisere de langsiktige konsekvensene av en action og policy, i stedet for å bruke kun umiddelbar reward.

4.2 Q-verdier

Verdifunksjon der Q står for *quality* og angir verdien (kvaliteten) til en state-action kombinasjon, som $Q(S, A)$. Vi beregner den ved bruk av Bellman-likningen:

$$Q^{new}(S_t, A_t) \leftarrow (1-\alpha) \cdot Q(S_t, A_t) + \alpha \cdot (R_{t+1} + \gamma \max_A Q(S_{t+1}, A))$$

- $Q^{new}(S_t, A_t)$ representerer den oppdaterte verdien av å være i S_t og gjøre A_t
- $Q(S_t, A_t)$ er den gamle verdien av å være i S_t og gjøre A_t. Hvis du bruker tabell initialiseres denne ofte til 0, og for nevralt nettverk har denne tilfeldige vekter i starten.
- R_{t+1} er reward mottatt for å være i S_t og gjøre A_t
- $\max_A Q(S_{t+1})$ er maksimal verdi av state S_{t+1} for alle tilgjengelige actions.
- α angir #læringsraten, hvor mye av gamle vs nye verdien til Q skal beholdes/påvirke parantesen.
- γ kalles #discount_factor og gjør at rewards mottatt tidligere vektes ulikt fra rewards mottatt senere.

Dette forteller oss at vi må beregne Q-verdien til et state-action-par iterativt, som fører til loopen:

1. Q initialiseres.
2. For hvert steg t har miljøet en tilstand S_t, agenten velger en action A_t, observerer en reward R_{t+1} og får en ny tilstand S_{t+1}.
3. Q oppdateres basert på disse verdiene og 2-3 gjentas til RL-loopen terminerer.

Forskjellen mellom verdi og reward er subtil men viktig.

- Verdifunksjonen (Q-verdiene) angir forventede verdien av en action i en state inkludert fremtidig verdi under en gitt policy.
- Reward får agenten fra environmentet som en tilbakemelding på agentens actions i ulike states.
  Vi får reward fra environment, mens Q må læres.

4.3 Environments

4.3 Environments

Det er fult mulig å lage egne miljøer for reinforcement learning; alt de må kunne er å ha en tilstand, ta imot en action, transisjonere til en ny tilstand og returnere reward. Likevel er det vanligste, særlig når man lærer RL for første gang, å bruke tilgjengelige treningsmiljøer. Blant de mest brukte er OpenAI's `Gym`, og det er vanlig å starte med klassiske kontrollproblemer, som `cartpole`, `mountaincar` og `pendulum`. `Gym`-APIet har også gitt opphav til det nærmeste vi kommer en standardfunksjonalitet for environments brukt i RL. Spesifikt er det vanlig å ha en funksjon som returnerer environmentet til en starttilstand, og som returnerer denne tilstanden samt eventuell ekstra info:

```
env.reset() → Tuple[initial_state, info]
```

For å lage RL-loopen trengs i utgangspunktet kun én ytterligere funksjon, nemlig

```
env.step(action) → Tuple[state, reward, done, info]
```

Funksjonen `.step` tar altså en action som argument, hvorpå environmentet havner i en ny tilstand, som returneres sammen med eventuell reward mottatt for denne action, en boolean `done` som angir om environmentet har terminert, og eventuell ekstra info. Basert på de to kommandoene over kan man kode en RL-loop. Først settes environment i starttilstanden, og deretter lages enten en `for`-loop som itererer over maks antall steg og terminerer når maks antall steg er nådd eller når `done==True`, eller en `while`-loop som terminerer når `done==True`. I denne loopen interagerer agenten med environmentet gjennom å sende inn actions, motta ny state og reward, og velge nye actions.

Som intuitivt eksempel kan vi se for oss spillet tre-på-rad. Starttilstanden `initial_state` er et brett med ni tomme ruter. Første spiller (menneske eller agent) plasserer en `'X'` på brettet, altså utfører en action. Dette gjør at environmentet inntar en ny state, med én rute okkupert av en `'X'`. Basert på denne staten plasserer den andre spilleren (menneske eller agent) en `'O'` på brettet, og environmentet inntar en ny state basert på denne. Slik fortsetter de to spillerne annenhver gang, frem til enten en av spillerne har fått tre av sine markører på rad (vertikalt, horisontalt eller diagonalt), eller til brettet ikke har flere tomme ruter igjen. I begge disse tilfellene settes `done=True`. Spilleren som får tre på rad vinner, og får `reward>0`. Hvis ingen får tre på rad er spillet uavgjort, og begge spillerne får `reward=0`.

Om vi vil skrive koden som lager dette environmentet, trenger vi minimum følgende funksjoner:

- `__init__`: Lager initial state, altså et tomt brett. Det kan for eksempel representeres som `'012345678'`. Setter `winner=None`, setter en variabel `player_turn='X'`. Kan eventuelt initialisere variabler som teller hvor ofte de ulike spillerne har vunnet, og andre ting som er nyttige å ta vare på.
- `reset`: Setter environmentet tilbake til initial state, `winner=None` og `player_turn='X'`, reinitialiserer eventuelle tellevariabler. Returnerer initial state og eventuelt en info-dictionary.
- `step`: Tar en action som argument og lager en ny state fra action, for eksempel `state='0123X5678'` hvis spiller `X` setter et kryss i midten. Sjekker om noen har vunnet, bytter på spillers tur (`X` hvis `O`, og omvendt). Returnerer ny state, reward, done og eventuelt info-dictionary.
- `check_winner`: Leter etter horisontale, vertikale eller diagonale linjer av `XXX` eller `OOO`. Returnerer `True` hvis det finnes en vinner, ellers `False`.
- `render`: Ikke nødvendig, men veldig nyttig om man vil se på environmentet under trening, eller når den ferdig trente agenten spiller. Lager en visuell representasjon av spillets tilstand.

Med et environment på plass – enten et vi har laget selv, eller et vi har importert fra et bibliotek – kan vi lage agenten som skal lære å løse en oppgave i environmentet.

4.4 Agenter: exploration, exploitation og læring

Agenten skal bruke Q verdier til å velge actions basert på states. Det kalles Q-learning og kan gjøres på flere måter. Lagres typisk som en dictionary med state som key.

Vi må bestemme #hyperparameter α og γ. Måten agenten får tilgang til treningsdata er gjennom å utføre ulike actions i ulike states. For å sikre at den utforsker de tilgjengelige mulighetene brukes hyperparameteren #exploration_factor, oft brukt $\epsilon \in [0, 1]$. Styrer hvor mye agenten utforskerer (explorer) miljøet sitt, og representerer sannsynligheten for at agenten velger en tilfeldig handling/action. Funksjonen som velger actions, skal velge actionen som tilsvarer høyest Q-verdi med en sannsynlighet på $1 - \epsilon$, ellers velge en tilfeldig action fra environmentets action space. Dvs. høy epsilon → Mye tilfeldige actions. Lav epsilon → Velger action med høyest Q-verdi.

Hvis agenten utforsker (explore) miljøet, som svarer til en høy verdi av ϵ, finner den kanskje kombinasjoner av states og actions som fører til store belønninger. På den annen side kan agenten utnytte (exploit) det den vet om miljøet, som svarer til en lav verdi av ϵ, og unngå risikoen for å gjøre actions som fører til store tap. Ulempen er at den da kanskje går glipp av belønninger dens aktuelle Q-verdier ikke vet om, fordi tilsvarende state-action-kombinasjon ikke har blitt besøkt før. Valget mellom utforsking og utnytting – som rommer faktumet at den ene går på bekostning av den andre, da de to svarer til henholdsvis høye og lave verdi av ϵ – kalles *exploration exploitation tradeoff*, og det finnes ingen universell løsning som gir en god tradeoff: Exploration handler om å ta actions med ukjent utfall, mens exploitation handler om å ta actions med kjent utfall. For å velge den optimale balansen mellom de to, måtte vi kjent utfallet til actions med ukjent utfall. Disse ville da per definisjon hatt kjent utfall, som er en selvmotsigelse. En mye brukt og ofte god strategi er *epsilon greedy*, med avtagende verdier av ϵ, såkalt *epsilon decay*. Da starter man med en høy verdi av ϵ tidlig i treningen, og reduserer denne stegvis. Dette fører til at agenten (nesten) slutter å gjøre tilfeldige actions sent i treningen, når den har lært seg mange av Q-verdiene. Når dette bør skje er avhengig av størrelsen på state-action-space, altså av environment og problemet som skal løses. Agenten slutter bare nesten å ta tilfeldige actions fordi det ofte fungerer bedre å beholde ϵ på en liten verdi, i stedet for å sette denne til nøyaktig 0, selv etter at treningen er avsluttet. Grunnen til det er simpelthen at selv en godt trent agent kan kjøre seg fast, og trenge et tilfeldig dytt for å komme seg videre.

Kan så lage en `Player` klasse får å gjøre agenten til et objekt. For tre-på-rad eksempel må en ha minimum

- Tag ← "X" eller "0"
- Q-tabell ← en dictionary, inneholder alle Q-verdiene agenten kjenner, og må læres
- α, γ, ϵ ← agentenes parametre. I tillegg kommer eventuelle læringsspesifikke parametre.

I tillegg må agenten ha metoder for å velge action, lære, og oppdatere ϵ. Vi kan bruke følgende kodesnutt for å velge action.

```
def choose_action(self, state):
    available_actions = [_i for _i, _s in enumerate(state) if _s.isnumeric()]
    # If only one move available, make that one
    if len(available_actions) == 0:
        return None
    elif len(available_actions) == 1:
        action = available_actions[0]
        return action
    else:
        qvalues = []
```

```
        for _action in available_actions:
            potential_state = state[:_action] + self.tag + state[_action+1:]
            qvalues.append(self.model.predict(self.state_to_array(potential_state),
                verbose=0)[0][0])
        max_index = np.argmax(qvalues)
        best_action = available_actions[max_index]
        return best_action
```

Denne funksjonen gjør flere ting: 1) Får oversikt over mulige trekk. `state` er en string bestående av indeksene til mulige trekk og eksisterende plasseringer, for eksempel `0123X0678`. Mulige trekk er alle delene av denne stringen som er numeriske verdier. Hvis det ikke finnes tilgjengelige trekk, returnerer funksjonen `None`, og hvis det kun finnes ett tilgjengelig trekk, returneres dette. 2) Sjekker om `state` finnes i Q-tabellen. Hvis ikke settes staten inn med default Q-verdi 0. 3) Trekker et tilfeldig tall mellom 0 og 1. Hvis dette er mindre enn ϵ, returneres en tilfeldig action. 4) Hvis det tilfeldige tallet er større enn ϵ, returneres action tilsvarende høyeste Q-verdi for aktuelle `state`, i følge Q-tabellen.

4.5 Q-tabell

Q-læring handler om å oppdatere Q-verdiene etter hvert som agenten får erfaring fra environmentet, som vil si fylle ut Q-tabellen gjennom læring. Vanlig å ha tom/0 i starten, også ha en nøstet dictionary som fylles ut etterpå med states som keys, også hver state har en egen dictionary med actions som keys for å state-action-parene for hver state.

I starten av læringen er agenten en komplett idiot og gjør tilfeldige trekk. På slutten av hvert spill får den en reward som brukes i læringsprosessen for å fylle inn og forbedre verdiene i Q-tabellen. Eks kode:

```
def learn(self, state, action, reward, new_state, done):
    v_s = self.calc_value(state)[0]
    if done:
        v_s_next = 0
    else:
        v_s_next = self.calc_value(new_state)[0]
    td_target = reward + (0 if done else self.gamma*v_s_next)
    td_delta = td_target - v_s
    new_qvalue = v_s + self.alpha*td_delta
    y_train = new_qvalue
    model_input = self.state_to_array(state).reshape(1,-1)
    self.train_model(model_input, y_train, self.train_steps)
    return
```

Først henter den ut gamle Q-verdien fra dictionary `v_s` $= (1-\alpha) \cdot Q(S_t, A_t)$ også henter ut maksimale fremtidige Q-verdien med nye staten om den eksisterer (0 hvis ikke) som `v_s_next` $= \max_A Q_{S_{t+1},A}$.
Deretter beregner den og erstatter gamle Q-verdien med den nye og trener modellen på nytt med staten den er i og nye Q-verdien.

Vi trenger funksjon for å oppdatere ϵ i løpet av treningen også:

```
def update_exp_factor(self):
    if self.exp_factor > self.exp_min:
        self.exp_factor *= self.epsilon_decay
```

Generelt sett med RL så om en agent ikke har rukket å lære noe innen $exp_factor > exp_min$ så kommer den ikke til å lære noe mer ved å trene videre og burde endre noe annet som tapsfunksjonen.

Kode for å tre-på-rad:
https://gitlab.com/Strumke/ntnu-lectures/-/blob/main/Tictactoe%20Q-learning/watch_tictactoe.py?ref_type=heads

Agenten får ikke reward før spillet er over, men gjennom Q-verdiene kan vi forstå agentens forventede fremtidige reward. Sagt mer presist representerer Q-verdiene forventet kumulativ fremtidig reward av å gjøre action A i state S og følge policyen deretter.

$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot \left[R(s,a) + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)\right]$$

4.6 Deep Q-learning

I stedet for Q-tabeller kan vi bruke et nevralt nettverk til å estimere Q-verdier, inkludert for tilstander agenten aldri har vært i. Om vi bruker dype nevrale nettverk, kaller vi det #deep_reinforcement_learning, forkortet DRL. Et dypt nevralt nettverk for å estimere Q-verdier kalles et #deep_Q-network, forkortet DQN.

4.6.1 Environment: Frozen lake

Resultatene i forelesningen er produsert ved hjelp av koden i følgende repo: gitlab.com/Strumke/ntnu-lectures/-/tree/main/Frozenlake%20DQN. Her brukes `Gym`-environmentet `FrozenLake`, se figur 31. Vi kan laste inn environmentet og se på state- og action-space ved bruk av følgende kode.

```
import gymnasium as gym
env = gym.make('FrozenLake-v1', map_name='4x4', is_slippery=False)
# legg til render_mode='human' for visualisering
print(env.action_space)
print(env.observation_space)
```

Vi vil da se at action space er diskret med 4 tilgjengelige actions, nemlig 0 (move left), 1 (move down), 2 (move right) og 3 (move up). Videre ser vi at observation space også er diskret, med 16 mulige verdier, som tilsvarer at agenten kan befinne seg på hver av de 16 rutene. Som tidligere setter vi environmentet til utgangstilstanden gjennom `env.reset()`, og vi tar steg ved bruk av `.step` med valgt action som argument. Dette environmentet returnerer `done=False` inntil agenten faller i et av hullene, har gjort flere enn 100 steg, eller når frem til julegaven som markerer mål. Hvis sistnevnte skjer returnerer environmentet `reward=1`, ellers er `reward=0`.



(a) (b)

Figure 31: Eksempler på visualiseringer av ulike states i Frozen Lake environment fra Gymnasium.

4.6.2 DQN: Q network og target network

Vi trenger 4 noder i output laget for å predikere Q-verdiene for hver action.
Agenten kan befinne seg i 16 uilike ruter, og da kan det være lurt å bruke one-hot-encode input. Bruker vi det kan vi ta inn en state som et tall [0,15] og gjøre om til en tensor med shape [1,16] bestående av den one-hot-encodede versjonen av samme state.
Eksempel kode:

```
def preprocess_state(state, num_states):
    # num_states = env.observation_space.n
    onehot_vector = F.one_hot(torch.tensor(state, dtype=torch.long), num_states)
    return onehot_vector


def create_model(self):
    return MLP(self.observation_space.n, self.action_space.n)
```

```
class MLP(nn.Module):
    def __init__(self, input, output):
        super().__init__()
        self._model = nn.Sequential(
            nn.Linear(input, 16),
            nn.ReLU(),
            nn.Linear(16, 32),
            nn.ReLU(),
            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Linear(16, output),
        )

    def forward(self, x):
        return self._model(x.float())
```

Funksjonen bør tilhøre klassen agenten er en instans av. Trener vi det nevrale nettverket til å løse Frozen Lake, går det nok dårlig. Grunnen er at vi har ikke en enket Q-verdi vi skal oppdatere som i Q-tabellen lenger, men oppdatere alle nettverksparametrene i DQN samtidig. Da blir alle Q-verdiene endret, også for actions i

neste state. Da får vi problemer for trening, siden vi ender opp å trene med et bevegelig mål som target. Vanligste konsekvensen av dette problemet er ustabil trening.

Et annet kjent fenomen innen RL er at agenten kan plutselig glemme det den har lært, calt #catastrophic_forgetting, som vises gjennom at nettverket først lærer seg å løse oppgaven og mottar høy reward over en periode, men så kollapser reward og nettverket begynner å gjøre dårlige prediksjoner. Dette kan skje selv om både læringsraten og ϵ har lave verdier.

- Lengre trening løser ikke dette, da den glemmer i takt med at det lærer.
- Skyldes som oftest overtilpasning til nylige hendelser mens gammel lærdom forsvinner i nye parameteroppdateringer.

Problemene som forårsakes av at det samme nettverker representerer policy og oppdateres underveis, og at nettverket overtilpasser seg til nylige hendelser kan løses ved hjelp av to virkemidler:

1. `target network`, et nytt nveralt nettverk som utgjør en stabilt mål og en stabil policy under treningen. Er en kopi av originale Q-nettverket som oppdateres med lavere frekvens og predikerer $\max_A Q$ i Bellman-likningen under trening. Oppdateres ved å kopiere vektene fra Q-nettverket ved en satt frekvens.
2. `replay buffer`, lagrer acions, states, nye states og rewards i minnet, som trekkes tilfeldig datapunkter fra under trening. Trekningen av datapunkter utgjør en batch.

```
class DQNAgent:
    def __init__(
        self,
        env: gym.Env,
        epsilon=0.999,
        epsilon_min=0.01,
        epsilon_decay=0.999,
        discount=0.95,
        batch_size=32,
        target_update_frequency=10,
        learning_rate=0.001,
        replay_size=1000,
    ):
        self.epsilon = epsilon
        self.epsilon_min = epsilon_min
        self.epsilon_decay = epsilon_decay
        self.discount = discount

        self.action_space = env.action_space
        self.observation_space = env.observation_space

        self.criterion = torch.nn.functional.mse_loss

        self.batch_size = batch_size
        self.replay_size = replay_size
        self.target_update_frequency = target_update_frequency

        self.replay_buffer = ExperienceReplayBuffer(
            self.replay_size, self.observation_space.n
        )

        self.filename = "model.pth"
        self.q_network = self.load_model()
        self.optimizer = torch.optim.Adam(self.q_network.parameters(), lr=learning_rate)

        self.target_network = self.create_model()
        self.target_network.load_state_dict(self.q_network.state_dict())
```

Kan gjenbruke `update_epsilon` & `select_action` fra Q-tabell koden ettersom det er samme. Da mangler bare selve læringsfunksjonen.

4.6.3: DRL: loss av Q-verdier og backpropagation

Funksjonen vi bruker for å trene Q-nettverket må

- hente data fra replay buffer,
- estimere maksimal verdi for neste states ved hjelp av target network,
- bruke $\max_A(Q)$ til å beregne oppdatert Q-verdi,
- finne predikert Q-verdi for valgte actions (selv om disse ble valgt gjennom exploration) fra Q- nettverket,
- beregne loss mellom predikert Q-verdi og oppdatert Q-verdi, og
- backpropagate loss gjennom, altså trene, Q-nettverket.

Å trene Q-nettverket gjøres ved å beregne loss på en batch, beregner gradienten til nettverksparametrene, og så oppdaterer parametrene basert på gradienten ved hjelp

av backpropagation.

Utfordringen er å beregne loss, siden vi har $Q(S_t, A_t)$ predikert, men vi ønsker at modellen faktisk predikerer $R_{t+1} + \max_A Q(S_{t+1}, A)$. Vi har følgende:

- En Q-verdi per action fra Q-nettverket som skal trenes, og
- oppdatert Q-verdi (target) for den ene action agenten utført.
  Vi kan da regne loss mellom den predikerte og oppdaterte Q-verdien for den ene action som ble valgt for å havne i neste state. Så kan vi bruke loss til backpropagation og trene Q-nettverket.

```
def learn(self):
    states, actions, rewards, next_states, dones = self.replay_buffer.sample(
        self.batch_size
    )

    with torch.no_grad():
        next_target_q_value = (
            self.target_network(next_states).max(dim=1, keepdim=True).values
        )
        next_target_q_value = torch.where(
            dones, torch.zeros_like(next_target_q_value), next_target_q_value
        )
        y_train = rewards + (self.discount * next_target_q_value)

    predicted_q = self.q_network(states)
    y_pred = torch.gather(predicted_q, 1, actions)

    self.optimizer.zero_grad()
    loss = self.criterion(y_train, y_pred)
    loss.backward()
    self.optimizer.step()
```

Da kan vi lage RL-loopen:

```
env = gym.make("FrozenLake-v1", map_name=f"4x4", is_slippery=False, render_mode="rgb_array")
agent = DQNAgent(env)

max_steps = 50
episodes = 3000
total_steps = 0

episode_steps = []

for ep in range(episodes):
    if ep % 100 == 0:
        print("Episode ", ep)

    state, _ = env.reset()
    state = preprocess_state(state, env.observation_space.n)
    done = False
    truncated = False

    for step in range(max_steps):
        total_steps += 1
        action = agent.select_action(state)

        next_state, reward, done, truncated, _ = env.step(action)
        next_state = preprocess_state(next_state, env.observation_space.n)

        agent.replay_buffer.add(state, action, reward, next_state, done)
        if agent.replay_buffer.size > agent.batch_size:
            agent.learn()

            if total_steps % agent.target_update_frequency == 0:
                agent.update_target_network()

        state = next_state

        if done or truncated:
            episode_steps.append(step)
            break

    agent.update_epsilon()
```